

# Pseudorandom Generators and Cryptography

Léa Machefel and Marilou Le Bricon

May 25, 2026

## Contents

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                | <b>3</b>  |
| <b>2</b> | <b>Generating randomness with a computer</b>                       | <b>4</b>  |
| 2.1      | True Random Number Generator (TRNG): . . . . .                     | 4         |
| 2.2      | Deterministic Random Bit Generator (DRBG) : . . . . .              | 4         |
| <b>3</b> | <b>Pseudorandomness in cryptography</b>                            | <b>6</b>  |
| 3.1      | Required security properties . . . . .                             | 6         |
| 3.2      | Statistical tests . . . . .                                        | 7         |
| <b>4</b> | <b>Pseudorandom Generators</b>                                     | <b>8</b>  |
| 4.1      | Generators based on Linear Recurrences . . . . .                   | 8         |
| 4.1.1    | Linear Congruential Generators [4] . . . . .                       | 9         |
| 4.1.2    | The PCG Generator used by NumPy . . . . .                          | 11        |
| 4.1.3    | Linear Feedback Shift Registers (LFSR) [7] . . . . .               | 12        |
| 4.1.4    | Generalized Feedback Shift Registers: The Mersenne Twister [6] . . | 13        |
| 4.2      | Generators based on hard mathematical problems . . . . .           | 15        |
| 4.3      | Generators based on cryptographic primitives . . . . .             | 17        |
| 4.3.1    | Hash-Based Generators: Hash-DRBG [12] . . . . .                    | 17        |
| 4.3.2    | Block-Cipher Generators . . . . .                                  | 19        |
| 4.3.3    | Stream-Cipher Generators . . . . .                                 | 20        |
| <b>5</b> | <b>Implementation</b>                                              | <b>23</b> |
| 5.1      | CTR-DRBG based on AES . . . . .                                    | 23        |
| 5.2      | Implementation Architecture . . . . .                              | 24        |
| 5.2.1    | Code organization and entropy sourcing . . . . .                   | 24        |
| 5.2.2    | The derivation function . . . . .                                  | 24        |
| 5.2.3    | Data structure and security bounds . . . . .                       | 24        |
| 5.2.4    | Core functions in <code>ctr_drbg.c</code> . . . . .                | 25        |
| 5.3      | Testing and Validation . . . . .                                   | 26        |
| 5.3.1    | Functional and Compliance Testing . . . . .                        | 26        |
| 5.3.2    | Statistical Randomness Testing . . . . .                           | 26        |
| 5.3.3    | Additional Empirical Tests . . . . .                               | 27        |
| 5.3.4    | Tests result & Interpretation . . . . .                            | 28        |
| <b>6</b> | <b>Conclusion</b>                                                  | <b>31</b> |

|          |                                            |           |
|----------|--------------------------------------------|-----------|
| <b>A</b> | <b>Appendices</b>                          | <b>34</b> |
| A.1      | CTR_DRBG Implementation . . . . .          | 34        |
| A.1.1    | Header File: ctr_drbg.h . . . . .          | 34        |
| A.1.2    | Source File: ctr_drbg.c . . . . .          | 34        |
| A.1.3    | Main Program: main.c . . . . .             | 36        |
| A.2      | Implementation of CTR_DRBG Tests . . . . . | 37        |
| A.2.1    | Tests File: tests.c . . . . .              | 37        |

# 1 Introduction

Randomness generation is a fundamental pillar of modern cryptography. The security of cryptographic protocols does not rely only on the deterministic properties of algorithms, but also on the unpredictability of certain secret or public values. In many settings, even a perfectly designed cryptographic scheme becomes insecure if the randomness it relies on is flawed or predictable.

Randomness plays a key role at several levels:

- **Key generation:** a cryptographic key must be sampled uniformly from a sufficiently large space to guarantee resistance against brute-force attacks. Any bias or predictability in the key generation process may immediately compromise security.
- **Nonces and initialization vectors (IV):** in symmetric encryption schemes (CBC, GCM, CTR, etc.), uniqueness—and in some cases unpredictability—is required to prevent structural weaknesses, replay attacks, or reuse of internal states.
- **Asymmetric cryptography:** generating primes for RSA or secret scalars for Diffie–Hellman relies on high-quality entropy. Insufficient randomness may lead to key recovery attacks.
- **Interactive protocols:** authentication schemes, identification protocols, and digital signatures often rely on random challenges or ephemeral secrets, whose predictability directly leads to forgery or impersonation attacks.

From a theoretical perspective, modern cryptographic security is formulated in terms of *indistinguishability* [1]: an adversary should not be able to distinguish a real execution of a protocol from an ideal one in which all random values are perfectly uniform [1]. This paradigm highlights that the quality of randomness is not merely a practical concern, but a central component of formal security guarantees.

## 2 Generating randomness with a computer

A computer is a deterministic system and cannot produce true randomness on its own; it must rely on external unpredictable physical phenomena [17].

- Thermal noise: caused by the random motion of electrons in a conductor due to temperature. It is commonly exploited in electronic circuits such as resistors or diodes to generate entropy.
- Shot noise: arises from the discrete nature of electric charge when current flows through a semiconductor junction. It is often used in hardware random number generators based on diodes.
- Clock jitter: refers to small, unpredictable variations in the timing of clock signals in digital circuits. Measuring these variations provides a source of entropy in many modern processors.
- Quantum phenomena: inherently random processes such as photon emission or splitting (e.g., through a beam splitter) provide high-quality entropy and are used in dedicated quantum random number generators.

These sources are measured through hardware sensors and digitized. However, raw outputs may contain bias or correlations, so they are typically processed through a conditioning function (e.g., a hash function) to produce uniform random bits [13].

### 2.1 True Random Number Generator (TRNG):

Definition:

A true random number generator (TRNG) is a device that extracts randomness from a physical process assumed to be non-deterministic.[13]

Characteristics:

- Based on non-deterministic physical phenomena
- Produces high-quality entropy but at a relatively low rate
- Requires dedicated hardware
- Output may be biased or noisy and requires post-processing
- Used primarily to initialize cryptographic systems

### 2.2 Deterministic Random Bit Generator (DRBG) :

Definition:

A pseudorandom number generator (PRNG) is a deterministic algorithm that produces a sequence of numbers that appears random, but is entirely determined by an initial value called a seed. [12]

Two main families of PRNG are usually distinguished:

- **Classical PRNGs**: mainly used for simulation and numerical applications; they are generally not secure,

- **Cryptographically secure PRNGs (CSPRNG)**: designed for cryptographic use.

A DRBG (Deterministic Random Bit Generator) is a standardized construction of a cryptographically secure pseudorandom generator specified by NIST in SP 800-90A[12]. This standard defines concrete algorithms for pseudorandom bit generation.

In this work, we focus on DRBGs.

#### Characteristics:

- Purely deterministic algorithm
- Expands a short, high-entropy seed into a long pseudorandom stream
- Very fast and suitable for high-throughput applications
- Reproducible if the seed is known (useful for testing or synchronization)
- Security relies on cryptographic assumptions

#### Examples:

- CTR-DRBG based on AES
- HMAC-DRBG based on SHA-256
- Hash-DRBG
- ChaCha20-based constructions (e.g., used internally in modern Linux RNGs)

#### **Practical interaction:**

In modern systems, TRNGs and DRBGs are combined: a TRNG provides an initial seed with sufficient entropy, which is then expanded by a DRBG to generate large amounts of pseudorandom data efficiently.

Modern operating systems such as Linux rely on hybrid entropy architectures combining hardware noise sources with cryptographic post-processing (notably ChaCha20-based extraction) to continuously reseed their random number generators.

This design follows the general recommendations of NIST regarding entropy collection and conditioning mechanisms for cryptographic random bit generation [12, 13].

This hybrid design ensures both strong unpredictability (from physical entropy) and high performance (from deterministic generation), and is recommended by current standards.

### 3 Pseudorandomness in cryptography

Not all PRNGs are suitable for cryptographic use. A classical pseudorandom generator (used in simulation or scientific computing) primarily aims to produce a good statistical distribution and a long period. However, these properties are not sufficient for cryptographic applications.

**Formal definition.** A pseudorandom generator (PRG) is a deterministic function

$$G : \{0, 1\}^n \rightarrow \{0, 1\}^{\ell(n)}$$

where  $\ell(n) > n$ , that expands a short uniformly random seed into a longer bitstring. [2] The generator is said to be secure if its output is computationally indistinguishable from a truly random string of length  $\ell(n)$ .

A generator is said to be *cryptographically secure* (CSPRNG) only if it produces a sequence statistically close to a random sequence and is robust against attacks: if part of the sequence is known, it must be impossible to reconstruct past outputs or predict future ones.

#### 3.1 Required security properties

For a PRG to be usable in cryptography, it must satisfy strict security properties.

**Computational indistinguishability.** Its output must be indistinguishable from a uniform random sequence for any polynomially bounded adversary: no efficient algorithm should be able to distinguish its outputs from truly random values [1].

**Next-bit unpredictability.** An equivalent formulation of security states that, given the first  $k$  output bits, no efficient adversary can predict the  $(k+1)$ -th bit with probability significantly better than  $1/2$ . This property, known as the *next-bit test*, is equivalent to computational indistinguishability [3].

**Unpredictability.** Observing a long portion of the output must not allow predicting future bits with probability significantly greater than random guessing [12].

**State reconstruction resistance.** It must be computationally infeasible to reconstruct the internal state from observed outputs, unlike many non-cryptographic generators.

**State compromise resistance.** If the internal state is compromised at some point, the adversary must not be able to reconstruct past outputs (backtracking resistance) nor predict all future outputs after reseeding (prediction resistance) [12].

**Entropy requirements.** The security of a PRG critically depends on the entropy of its initial seed. This is typically quantified using *min-entropy*, defined as

$$H_\infty(X) = -\log_2 \left( \max_x \Pr[X = x] \right).$$

A high min-entropy ensures that the seed is sufficiently unpredictable, which is essential for the security of the generated sequence. [13]

## 3.2 Statistical tests

To evaluate the statistical properties of PRGs, several empirical test suites have been developed. These tests do not prove cryptographic security, but are essential for detecting statistical biases, correlations, or structural flaws in the generated sequences.

**The NIST SP 800-22 Suite.** The *NIST SP 800-22 Statistical Test Suite* [15] is a foundational standard defined to evaluate randomness. These tests aim to verify whether a binary sequence exhibits properties consistent with an independent uniform distribution. They evaluate in particular:

- the overall balance between 0s and 1s (Frequency Test);
- frequency within blocks;
- lengths of consecutive identical bits (Runs Test);
- linear complexity (Linear Complexity Test);
- presence of periodic structures via Fourier transform;
- compressibility (Approximate Entropy, Serial Test).

**The TestU01 Suite.** [16] While the NIST suite is widely used, the *TestU01* software library is currently considered the most rigorous tool for empirical PRNG testing. TestU01 provides multiple batteries of tests like *SmallCrush*, *Crush*, and *BigCrush*. These batteries execute a vast array of complex, multidimensional statistical tests (such as matrix rank tests, collision tests, and random walk evaluations) over extremely large sequences of data—BigCrush alone consumes roughly  $2^{38}$  bits. Generators that easily pass the NIST suite frequently exhibit systematic failures when subjected to the highly demanding tests within BigCrush.

**Methodology.** Across these suites, the general testing methodology remains similar: each test computes a statistic from the observed sequence and derives a *p-value*. If this p-value falls outside a predefined acceptance interval, the null hypothesis “the sequence is random” is rejected. Tests are applied to multiple independent sequences to evaluate the expected success rate statistically.

**Limitations of the tests.** It is critical to emphasize that passing empirical tests—whether NIST SP 800-22 or TestU01’s BigCrush—does not imply that a generator is cryptographically secure. These tests only evaluate observable statistical properties, not resistance against adversarial attacks or state reconstruction.

A well-parameterized linear generator may exhibit excellent multidimensional equidistribution and pass these tests while remaining entirely predictable. Standard cryptographically secure generators (such as CTR-DRBG, or Hash-DRBG) naturally pass all these suites when correctly implemented.

Thus, statistical suites like NIST and TestU01 are indispensable validation tools for simulation and basic sanity checking, but they do not replace a rigorous mathematical security analysis based on unpredictability and the robustness of the underlying cryptographic primitives.

## 4 Pseudorandom Generators

PRGs can be divided into several major categories depending on their construction and theoretical foundations. Historically, the earliest and most widely used generators rely on linear mathematical recurrences. A second category of generators bases its design on the computational hardness of complex mathematical problems. Currently, modern systems primarily rely on a third category: generators built from well-analyzed cryptographic primitives—such as hash functions or block ciphers which successfully balance high efficiency with rigorous security guarantees.

### 4.1 Generators based on Linear Recurrences

The earliest and most widely used conventional PRGs rely on linear mathematical recurrences to update their internal states. Because these generators use simple, linear operations such as modular addition and multiplication, or binary XOR combinations, they are extremely efficient and easy to implement in both hardware and software.

A defining advantage of linear generators is their mathematical tractability. Using techniques from linear algebra and finite field theory, it is possible to rigorously prove their statistical properties, such as guaranteeing a maximal period before the random sequence repeats.

However, this mathematical simplicity is also their fatal flaw. Because the state transitions are linear, an observer who collects a sufficient number of outputs can easily set up a system of linear equations to reconstruct the entire internal state. As a result, purely linear generators offer absolutely no cryptographic security.

This section explores the most fundamental linear structures: generators operating over modular arithmetic rings (such as LCGs and their permuted variants) and generators operating over binary finite fields (such as LFSRs).

Although these constructions are still widely used in simulation, statistical sampling, and non-security-critical applications, they are considered obsolete for cryptographic purposes due to their linear structure, which enables efficient state recovery attacks.



#### 4.1.1 Linear Congruential Generators [4]

Linear congruential generators (LCGs) are among the oldest and simplest families of PRGs. They were widely used in early software libraries and are still present in some implementations for simulation purposes.

The principle is based on a modular recurrence relation:

$$x_{n+1} = (ax_n + c) \bmod m$$

where:

- $x_n$  is the internal state at iteration  $n$ ;
- $a$  is the multiplier;
- $c$  is the increment;
- $m$  is the modulus.

The pseudorandom sequence is obtained directly from the values  $x_n$  or from a simple transformation (e.g., normalizing  $x_n/m$  to obtain a real number in  $[0, 1]$ ).

**Period of the generator** The maximum period of an LCG is equal to the modulus  $m$ . However, this maximum is only achieved if the parameters  $a$ ,  $c$ , and  $m$  satisfy specific conditions. The quality of the generator depends entirely on the choice of these parameters.

When  $c \neq 0$ , the Hull-Dobell theorem gives necessary and sufficient conditions for achieving maximal period:

- $c$  must be coprime with  $m$ ;
- $a - 1$  must be divisible by all prime factors of  $m$ ;
- if  $m$  is divisible by 4, then  $a - 1$  must also be divisible by 4.

These conditions ensure that the sequence cycles through all residues modulo  $m$  before repeating.

#### Advantages

- they are extremely fast to compute;
- they are very simple to implement;
- they require very little memory, as the internal state is typically a single integer.

**Cryptanalysis and State Recovery** Because the state transitions are strictly linear modulo  $m$ , an attacker can efficiently recover the internal parameters and predict all future states, even from a small sequence of observed outputs.

Let  $\{x_0, x_1, x_2, \dots, x_k\}$  be a sequence of consecutive outputs. Depending on the implementation, the cryptanalysis proceeds in two main scenarios:

### 1. Full Output State Recovery

If the outputs expose the full internal state  $x_n$ , the unknown parameters  $m$ ,  $a$ , and  $c$  can be recovered using a deterministic algebraic algorithm:

- **Eliminate  $c$ :** Compute the differences of consecutive outputs,  $d_n = x_{n+1} - x_n$ . This yields  $d_n \equiv a \cdot d_{n-1} \pmod{m}$ .
- **Recover  $m$ :** Construct the determinant sequence  $t_n = d_{n+2}d_n - d_{n+1}^2$ . Because of the linear dependence,  $t_n \equiv 0 \pmod{m}$  for all  $n$ . The modulus  $m$  (or a small multiple of it) is simply extracted by computing  $\gcd(t_0, t_1, \dots, t_{k-3})$ .
- **Recover  $a$  and  $c$ :** Once  $m$  is known, the multiplier is found via modular inversion:  $a \equiv d_1 \cdot d_0^{-1} \pmod{m}$ . The increment follows as  $c \equiv x_1 - a \cdot x_0 \pmod{m}$ .

### 2. Truncated Output State Recovery (Hidden State)

In practice, implementations often attempt to hide the state by outputting only the most significant bits of  $x_n$ . In this scenario, the state can be expressed as  $x_n = y_n \cdot 2^j + z_n$ , where the high bits  $y_n$  are public and the low bits  $z_n$  are hidden. Because the sequence of truncated outputs can be mathematically modeled as points on a discrete grid, the state recovery problem translates to finding a specific short vector in a lattice. The LLL (Lenstra–Lenstra–Lovász) algorithm resolves this by acting as an integer-adapted Gram-Schmidt process. It efficiently produces a "nearly orthogonal" basis composed of remarkably short vectors. In the context of a truncated LCG, the shortest vector in this reduced basis perfectly isolates the hidden bits  $z_n$ , completely revealing the generator's internal state.

**Complexity and Practical Feasibility** The feasibility of these attacks in practice is absolute, making LCGs entirely broken for cryptographic purposes:

- **Full Outputs:** The attack requires only 4 to 6 outputs. The computational complexity is dominated by the Euclidean algorithm for the GCD and modular inverse, requiring  $O(\log^2 m)$  bit operations. On modern hardware, this executes in a fraction of a millisecond.
- **Truncated Outputs:** The LLL algorithm operates in polynomial time, typically  $O(k^5 \log^3 m)$  where  $k$  is the dimension of the lattice (the number of observed outputs). For typical moduli (e.g., 32-bit or 64-bit), reconstructing the hidden state requires observing only a handful of outputs ( $k \approx 10$ ) and runs in milliseconds on standard hardware.

This makes LCGs a classical example of linear predictability in modular arithmetic.

### 4.1.2 The PCG Generator used by NumPy

The NumPy Python library uses a PRNG from the family of *Permuted Congruential Generators* (PCG), specifically the strongest variant known as PCG64.

A PCG generator combines two main steps to retain the simplicity and speed of a Linear Congruential Generator while improving output quality [8]:

1. **Updating the internal state:** The generator operates as an LCG modulo  $2^n$ . The state transition is defined as:

$$S_{i+1} = (aS_i + c) \mod 2^n$$

In the case of PCG64,  $n=128$ . The choices of the multiplier ( $a$ ) and the increment ( $c$ ) dictate the sequence's properties:

- **The Multiplier ( $a$ ):** In PCG64,  $a$  is a fixed 126-bit constant. If you think on the stream generated as coordinates of points, you can see that the choice of the multiplier directly dictates the spatial distribution of the generated sequences. All pure LCGs suffer from the “Marsaglia effect,” an inherent mathematical defect where generated multi-dimensional coordinates inevitably fall onto a discrete set of parallel hyperplanes. If  $a$  is chosen poorly, the generator produces very few planes with massive, predictable gaps of empty space between them. By carefully engineering  $a$ , PCG64 maximizes the number of these hyperplanes. This makes the planes dense and tightly packed, effectively minimizing the empty gaps and allowing the underlying LCG to provide a strong statistical foundation.
  - **The Increment ( $c$ ):** The increment  $c$  is user-definable (though a default exists) and its primary purpose is to provide several independent pseudorandom streams from the same seed, functioning similarly to an initialization vector (IV) in a stream cipher. For the LCG to achieve a maximum period,  $c$  must be an odd number.
2. **permutation:** To mask the LCG's underlying linearity, the generator applies a permutation to the output. In the case of PCG64, this function XORs the two 64-bit halves of the internal state together and rotates the result to the right by a data-dependent amount derived from the 6 most significant bits of the state. The output sequence  $X_i$  is computed as:

$$X_i = (S_i[0 : 64] \oplus S_i[64 : 128]) \gg S_i[122 : 128]$$

These generators are very popular in scientific simulation because they offer excellent performance, simple implementation, and good statistical properties. However, they are not designed to be cryptographically secure.

**Cryptanalysis of PCG64 [9]** Despite the use of a permutation, PCG64 remains vulnerable to algebraic cryptanalysis. Recent work demonstrated a practical algorithm that reconstructs the full internal state and hidden parameters, allowing the pseudorandom stream to be predicted deterministically [9].

The attack relies on a “guess-and-determine” strategy combined with lattice-based techniques. Because the underlying LCG uses a power-of-two modulus, the lower bits of

the state are completely unaffected by the higher bits. By guessing a small subset of the state's least significant bits and a few consecutive 6-bit rotation values, the attacker can expose a truncated version of the underlying LCG.

Reconstructing the state of a truncated LCG can be framed as finding a point in a Euclidean lattice, which requires solving an instance of the well-known Closest Vector Problem (CVP). The complexity of this lattice attack depends heavily on whether the increment  $c$  is known:

- **Scenario A: Known Increment.** When the default, known increment is used, the state reconstruction is highly efficient. The attacker guesses approximately 37 bits of the internal state (the lower bits and consecutive rotations). For each guess, they solve an instance of CVP in a small 3-dimensional lattice using Babai's rounding algorithm. This requires only 192 bits of output (three consecutive 64-bit outputs) and takes roughly 20 CPU minutes to reconstruct the original 128-bit seed.
- **Scenario B: Secret Increment.** By default in NumPy, both the state and the increment are initialized using an external source of entropy, making  $c$  unknown. This dramatically increases the attack's difficulty. The attacker must guess between 51 and 55 bits and solve millions of 4-dimensional CVP instances to recover 64 bits of information regarding the mathematical difference between consecutive internal states. Strong consistency checks are used to filter out bad guesses early on. Finally, the full state and secret increment are reconstructed by solving a 63-dimensional CVP instance. This robust approach requires 4096 bits of continuous output and averages 20,000 CPU hours, proving that state recovery is still practically feasible on computing clusters.

Despite additional permutations, the underlying linear structure remains exploitable.

#### 4.1.3 Linear Feedback Shift Registers (LFSR) [7]

Linear Feedback Shift Registers are a class of PRGs based on linear recurrences over the finite field  $F_2$ . An LFSR consists of a shift register of length  $n$  storing a binary state  $(s_0, s_1, \dots, s_{n-1})$ . At each clock cycle, the bits shift one position to the right, outputting the rightmost bit. To fill the empty position on the left, a new input bit is computed as a linear combination (XOR) of selected bits of the current state.

This feedback wiring is defined by a characteristic polynomial:

$$P(X) = X^n + c_{k_1}X^{k_1} + \dots + c_{k_r}X^{k_r} + 1$$

Rather than an equation to solve, this polynomial acts as a hardware blueprint. The highest degree  $n$  defines the total size of the register, while the non-zero coefficients  $c_{k_i} \in \{0, 1\}$  indicate exactly which bit positions are "tapped" and fed into the XOR gate.

The sequence produced by the LFSR is entirely deterministic and relies on two factors:

1. **The Initial State (Seed):** The starting bits must be provided by an external source of entropy. The only restriction is that the state cannot be entirely composed of zeros, because an all-zero state permanently locks the generator.
2. **The Feedback Polynomial:** If the chosen polynomial is *primitive* over  $F_2$ , the feedback wiring is optimal. This guarantees the LFSR achieves a maximal period of  $2^n - 1$ , meaning it will cycle through every possible non-zero state exactly once before the sequence repeats.

To make this concrete, imagine a simple 4-bit LFSR ( $n = 4$ ) with the characteristic polynomial  $P(X) = X^4 + X + 1$ . This polynomial indicates that the feedback "taps" are placed at the 1st and 4th positions. If the initial state is  $(1, 0, 0, 0)$ , the register computes the next incoming bit by evaluating  $1 \oplus 0 = 1$ . In the next iteration, the bits shift to the right, the rightmost 0 becomes the first output bit, and the newly calculated 1 enters the left side, updating the state to  $(1, 1, 0, 0)$ . Because this specific polynomial is primitive, the register will cycle through all  $2^4 - 1 = 15$ .

LFSRs are particularly attractive due to their simplicity and efficiency: they are extremely fast, require very little memory, and are easy to implement in both hardware and software. From a statistical standpoint, LFSRs with primitive polynomials exhibit excellent basic properties: they are perfectly balanced over a full period and can easily pass foundational tests like NIST SP 800-22. For these reasons, they are widely used in communication systems and error-correcting codes.

**Cryptanalytic Limitations** Despite these ideal statistical properties, LFSRs suffer from a fatal flaw in adversarial contexts: their strict linear algebraic structure. Because of this inherent linearity, the sequence is highly predictable. Given a sequence of output bits equivalent to only twice the size of the register ( $2n$ ), the Berlekamp–Massey algorithm can efficiently reconstruct the underlying polynomial and expose the entire internal state.

This linear structure over  $F_2$  directly enables algebraic reconstruction attacks.

#### 4.1.4 Generalized Feedback Shift Registers: The Mersenne Twister [6]

Introduced in 1997, the Mersenne Twister is a widely used PRNG that serves as an optimized, word-based evolution of Linear Feedback Shift Registers (LFSRs). While classical LFSRs operate on single bits, the Mersenne Twister processes full words (typically  $w = 32$  or  $w = 64$  bits), making it extremely fast in software environments.

The generator is named after its massive period length, which is chosen to be a **Mersenne prime**. A Mersenne prime is a prime number of the form  $2^p - 1$ , where  $p$  is also a prime.

To understand how the Mersenne Twister operates, its mathematical mechanics can be broken down into three distinct phases:

**1. The State Array and Initialization (Seeding)** The generator maintains an internal state array of  $n$  words, denoted  $(x_0, x_1, \dots, x_{n-1})$ . Before generating any numbers, this array must be properly populated from a starting seed. A dedicated initialization algorithm spreads the single seed value across the entire array:

$$x_i = f \cdot (x_{i-1} \oplus (x_{i-1} \gg (w - 2))) + i$$

This step ensures that even simple or similar seeds (e.g., 1 and 2) produce different, well-distributed initial states. It specifically prevents "zero-heavy" initial states, which would otherwise force the generator into a long phase of producing non-random outputs.

**2. The Twist (State Update)** Once the initial state array is populated, the generator produces new words by iterating through the array. At each step  $k$ , the generator updates the state using a linear recurrence over the finite field  $F_2$ :

$$x_{k+n} = x_{k+m} \oplus \left( (x_k^u \mid x_{k+1}^l) A \right)$$

Despite the complex notation, this represents a highly efficient digital mixing process:

- **Splitting:** The algorithm takes the upper bits ( $x_k^u$ ) of the current word and the lower bits ( $x_{k+1}^l$ ) of the next word. The parameter  $r$  defines exactly how many bits belong to the "upper" and "lower" halves.
- **Concatenation:** It glues these two partial words together ( $x_k^u \mid x_{k+1}^l$ ) to form a new intermediate word.
- **The Twist Matrix ( $A$ ):** This combined word is multiplied by a constant matrix  $A$ . Because  $A$  is carefully structured in rational normal form, this complex-sounding multiplication is optimized in software into a single fast bit-shift and a conditional bitwise XOR.
- **Feedback:** Finally, this result is XORed ( $\oplus$ ) with a word located  $m$  steps away in the array ( $x_{k+m}$ ), and this final value replaces the old  $x_k$  in the array.

**3. Tempering (Output Extraction)** You might expect the newly calculated word to be output directly to the user, but it first undergoes a final transformation known as *tempering*. Tempering consists of four sequential bitwise XORs and shifts applied to the word. Reading the raw words directly from the state array would expose the underlying linear algebraic patterns. Tempering mathematically scrambles the bits of the word right before it is output, drastically improving the statistical equidistribution without permanently altering the generator's internal state array.

The most common variant, MT19937, uses a state of  $n = 624$  words of 32 bits and achieves a period of  $2^{19937} - 1$ . It is known for its excellent statistical properties, including equidistribution up to 623 dimensions, and its very high performance due to the exclusive use of simple bitwise operations.

For these reasons, the Mersenne Twister has become a standard generator in many programming environments (e.g., Python, C++). However, despite its excellent statistical behavior, it is not suitable for cryptographic use. Indeed, the generator is entirely linear over  $F_2$ , and the tempering transformation is invertible. As a result, observing 624 consecutive outputs is sufficient to reconstruct the full internal state. Once the state is known, all future outputs can be predicted, and past outputs can also be recovered.

The large state size does not prevent full reconstruction due to invertible linear dynamics.

**Assessment of the subsection:** This section highlights that linear pseudorandom generators, despite their strong statistical properties and computational efficiency, rely on algebraic structures that inherently compromise unpredictability in adversarial settings. As a result, they are unsuitable for cryptographic applications and remain primarily relevant for simulation and non-security-critical tasks.

## 4.2 Generators based on hard mathematical problems

Another approach to designing PRGs is to base their security on the computational hardness of well-studied mathematical problems. An important example is the *Blum–Blum–Shub* (BBS) generator [5], which provides strong theoretical security guarantees.

**Construction and Mechanism** The construction begins by selecting two large prime numbers  $p$  and  $q$  such that:

$$p \equiv q \equiv 3 \pmod{4},$$

and defining the modulus:  $n = pq$ .

An initial seed  $x_0$  is chosen such that it is coprime to  $n$ , and the generator evolves according to the recurrence:

$$x_{i+1} = x_i^2 \bmod n.$$

At each iteration, one or more bits are extracted from  $x_i$  (typically the least significant bit) to form the output sequence.

**Security Basis** The core idea behind BBS is that its security reduces formally to the difficulty of factoring the modulus  $n$ . Because  $n$  is a Blum integer (the product of two distinct primes congruent to 3 (mod 4)), the squaring function  $f(x) = x^2 \bmod n$  acts as a permutation over the group of quadratic residues modulo  $n$ , denoted  $QR_n$ . Every  $x \in QR_n$  has exactly four square roots modulo  $n$ , but the Blum integer property guarantees that exactly one of these roots is itself in  $QR_n$ . This unique root is called the principal square root. By Rabin's theorem, computing this principal square root modulo a composite  $n$  is computationally equivalent to factoring  $n$ .

**Bit Extraction and Hard-Core Predicates** To translate this structural hardness into a secure bit sequence, the construction relies on the theory of *hard-core predicates*. Intuitively, a hard-core predicate is a specific bit of information about a hidden number that is mathematically proven to be just as hard to compute as the entire number itself. For the BBS generator, a celebrated theorem by Alexi, Chor, Goldreich, and Schnorr proves that the least significant bit (LSB) of  $x_i$  is a hard-core predicate for the modular squaring function.

To understand the significance of this, we can view their proof as a cryptographic reduction. Suppose an attacker discovers a slight statistical vulnerability. Specifically, assume they build an algorithm that, given the public value  $x_{i+1}$ , can guess the hidden LSB of the pre-image  $x_i$  with a probability of  $1/2 + \epsilon$  (meaning it performs just marginally better than a random coin toss). The theorem proves that if such an algorithm exists, it can be mathematically leveraged as a subroutine (an oracle) to completely reconstruct the entire hidden number  $x_i$ . By querying this "LSB-guessing algorithm" multiple times with cleverly manipulated mathematical inputs, one can efficiently piece together every single bit of  $x_i$ , thus successfully computing the principal square root.

This establishes a logical chain: because we know that computing the principal square root is mathematically equivalent to factoring the modulus  $n$ , it strictly follows that gaining even a slight statistical advantage in predicting a single LSB is also equivalent to factoring  $n$ . Therefore, as long as factoring  $n$  remains computationally infeasible, the individual output bits of the BBS generator are mathematically guaranteed to be unpredictable.

**Sequence Indistinguishability and the QRP** Furthermore, evaluating the sequence as a whole, the ability to distinguish the output of a BBS generator from a truly random uniform binary sequence is computationally equivalent to solving the Quadratic Residuosity Problem (QRP). The QRP asks to decide whether a given integer  $a \in Z_n^*$  with a Jacobi symbol  $(a|n) = 1$  belongs to  $QR_n$ . Since no efficient algorithm is known for solving QRP without knowing the prime factors  $p$  and  $q$ , the output sequence is provably unpredictable. This contrasts sharply with simpler generators such as LCGs or LFSRs, whose structure can be exploited directly.

**Practical Limitations** However, this strong security guarantee comes at a significant cost. Each iteration requires a modular squaring operation on large integers (typically hundreds or thousands of bits), making the generator slower than practical alternatives. Furthermore, only a small number of bits can be safely extracted per iteration without weakening the security guarantees, which further reduces its efficiency.

Because of these limitations, Blum–Blum–Shub is rarely used in real-world systems and is mainly of theoretical interest, serving as a benchmark for provable security in pseudorandom generation. In practice, modern cryptographic systems favor generators that achieve a better balance between security and efficiency by relying on well-analyzed cryptographic primitives such as hash functions or block ciphers.



### 4.3 Generators based on cryptographic primitives

As demonstrated by the strict vulnerabilities of linear algebraic generators (such as the Mersenne Twister) and the severe performance bottlenecks of number-theoretic constructions (such as Blum-Blum-Shub), practical software engineering requires a delicate balance. A modern operating system requires a generator that is fast, yet mathematically secure enough to resist attacks.

To bridge this gap, modern cryptographic architectures construct Deterministic Random Bit Generators by repurposing standard, highly optimized cryptographic primitives—specifically hash functions, block ciphers, and stream ciphers.

The fundamental intuition behind this approach is straightforward: if a cryptographic primitive possesses sufficient "confusion and diffusion" to securely mask a plaintext message (in the case of ciphers) or compress arbitrary data irreversibly (in the case of hashes), it is capable of securely hiding the evolving internal state of a PRG.

This methodology offers several critical advantages:

- **Hardware Acceleration:** Primitives like AES and SHA-256 benefit from dedicated hardware instructions on modern CPUs (e.g., AES-NI and ARM Cryptographic Extensions)
- **Cryptanalytic Maturity:** Rather than inventing new mathematical structures that might harbor hidden flaws, these generators rely on primitives that have already survived decades of cryptanalysis.
- **Standardized Security Proofs:** Frameworks such as the NIST SP 800-90A standard define rigorous, mathematically proven architectures that guarantee crucial properties like forward secrecy and backtracking resistance.

Today, this paradigm represents the standard for secure pseudorandomness.

#### 4.3.1 Hash-Based Generators: Hash-DRBG [12]

Hash-based generators use cryptographic hash functions (such as SHA-256 or SHA-3) to mimic randomness. A secure hash function is designed to be a one-way mathematical transformation; it deterministically compresses an input into a fixed-length output that is uniformly distributed and computationally indistinguishable from true randomness.

In the context of a Deterministic Random Bit Generator (DRBG), the security of the construction rests almost entirely on the hash function's pre-image resistance. This means that even if an attacker observes a continuous stream of generated output bits, it remains computationally infeasible to reverse the hashes and recover the generator's secret internal state. The most prominent standardized example of this architecture is **Hash-DRBG**, defined by NIST in SP 800-90A.

**Internal State and Parameters** The architecture relies on a fundamental parameter known as *seedlen* (seed length), which defines the maximum capacity of the internal state. Its value is dictated by the chosen hash function (for example, when using SHA-256, *seedlen* = 440 bits).

Hash-DRBG maintains a strictly managed internal state composed of three elements:

- **The Working State ( $V$ ):** A *seedlen*-bit integer initially populated by a high-entropy seed from a True Random Number Generator (TRNG).

- **The Constant ( $C$ ):** A *seedlen*-bit value derived by hashing the initial seed. It remains static between reseeds and ensures that state transitions remain algebraically complex.
- **The Reseed Counter:** An integer tracking the number of requests since the last initialization, used to enforce mandatory reseeding limits.

## Algorithm

- **1. Bit Generation:** To generate a requested sequence of bits, the algorithm temporarily increments  $V$  and concatenates the resulting hashes until the desired length is reached:

$$\text{Output} = \text{Hash}(V) \parallel \text{Hash}(V + 1) \parallel \text{Hash}(V + 2) \dots$$

- **2. State Update:** Once the output is delivered, the internal state  $V$  must be mathematically updated to ensure forward secrecy (preventing an attacker who compromises  $V$  in the future from calculating previous outputs). NIST specifies the following arithmetic update:

$$H = \text{Hash}(0\text{x}03 \parallel V)$$

$$V \leftarrow (V + H + C + \text{counter}) \bmod 2^{\text{seedlen}}$$

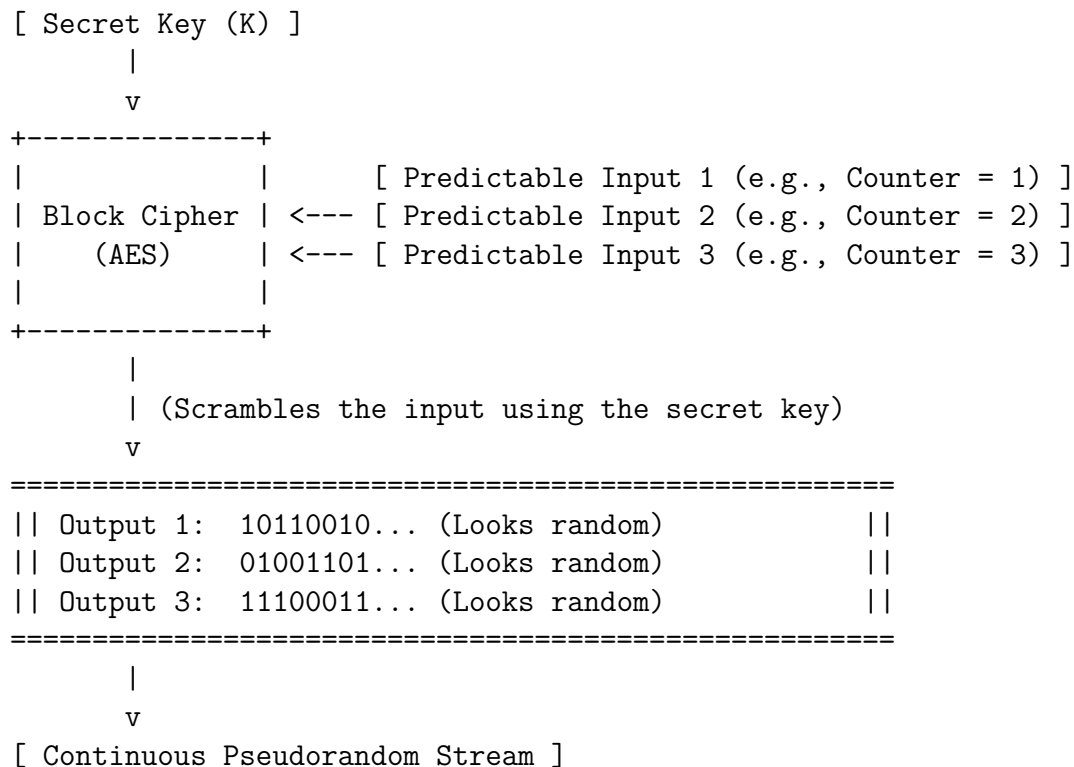
By combining a hash of the current state, the static constant  $C$ , and the incrementing counter via modular addition, the algorithm mathematically guarantees a long period and prevents the generator from collapsing into short, repeating cycles. The counter is then incremented ( $\text{counter} \leftarrow \text{counter} + 1$ ).

**Security and Performance** Its security reduces directly to the preimage, second-preimage, and collision resistance of the chosen hash function. When properly seeded, it provides excellent security guarantees, including absolute backtracking resistance.

- **Advantages:** Hash-DRBG is conceptually simple to implement, highly portable, and does not strictly require specialized hardware. Because hash functions are ubiquitous in cryptography, it is widely deployed in security modules and operating systems where a reliable source of pseudorandomness is required.
- **Limitations:** Historically, hash functions were slower to compute than block ciphers, giving AES-based generators (like CTR-DRBG) a performance advantage on platforms with AES-NI instructions. However, this gap has narrowed significantly with the introduction of hardware-accelerated SHA instructions on modern ARM and processors. As with all deterministic generators, its absolute security is entirely dependent on proper initial seeding and the strict enforcement of reseeding intervals.

### 4.3.2 Block-Cipher Generators

The fundamental principle behind block-cipher-based generators is the transformation of a fixed-length cryptographic primitive into a variable-length pseudorandom stream. A block cipher, such as AES, is designed to be a *Pseudorandom Permutation* : it takes a plaintext and a secret key to produce a ciphertext that is computationally indistinguishable from a random bitstring. In a DRBG construction, the roles are shifted: instead of encrypting a secret message, the cipher is used to encrypt a deterministic and often predictable input (like a counter or a timestamp). Because the block cipher effectively "scrambles" its input using a secret key, the resulting outputs appear as independent uniform distributions to any observer who does not possess the key.



Before the standardization of CTR-DRBG, the ANSI X9.31 generator (and its predecessor X9.17) was the primary block-cipher-based PRNG used in government-certified products. It provides a historical contrast to modern designs by demonstrating how subtle implementation flaws can compromise a theoretically sound algorithm.

**Operational Mechanics** Unlike modern constructions that rely on a simple incrementing counter, the legacy ANSI X9.31 standard [10] uses a date/time vector (*DT*) as an auxiliary input to prevent the generator from falling into short, predictable cycles.

At the core of this generator is the function  $E_K$ , which represents the encryption operation of a standardized block cipher (historically Triple-DES, and later AES). The fixed secret key  $K$  used by this cipher is established during the generator's initialization phase. This key must be drawn from an approved high-entropy source. The internal state is initialised with a TRNG.

Using this block cipher engine, the internal state  $V_i$  is updated in a three-step encryption process for each generation request:

1. A temporary value is generated by encrypting the current timestamp:

$$I = E_K(DT)$$

2. The pseudorandom output block  $R_i$  is computed by XORing the intermediate value with the current state, and encrypting the result:

$$R_i = E_K(I \oplus V_i)$$

3. The state is updated for the next round by XORing the newly generated output with the intermediate value, and encrypting once more:

$$V_{i+1} = E_K(R_i \oplus I)$$

**Vulnerabilities: The DUHK Attack [11]** The security of ANSI X9.31 assumes that both the state  $V$  and the key  $K$  remain secret. However, because the standard did not explicitly mandate that  $K$  be freshly generated for each device, many manufacturers hard-coded the same AES key into thousands of devices.

In 2017, researchers introduced the **DUHK (Don't Use Hard-coded Keys) attack**, a state-recovery attack that targets these implementations. If an attacker knows the hard-coded key  $K$  (e.g., by extracting it from a firmware update) and observes just 16 bytes of raw output (like a random nonce in a VPN handshake), they can:

- Brute-force the timestamp  $DT$  (which has relatively low entropy at microsecond granularity).
- Reconstruct the internal state  $V_i$ .

## Practical Applications and Constraints

- **Typical Software:** Historically, this generator was found in FIPS-certified VPNs, routers, and firewalls). Due to its vulnerabilities, NIST officially removed it from the list of approved generators in 2016.
- **Engineering Constraints:** The main constraint is the absolute requirement for key and seed unpredictability. ANSI X9.31 lacks this "backtracking resistance," meaning a single state compromise can reveal all previous random numbers generated since the last reseed.

### 4.3.3 Stream-Cipher Generators

Unlike block ciphers, stream ciphers are natively designed to generate a continuous, arbitrary-length sequence of pseudorandom bits (a keystream). Because their core function is literally expanding a short secret into a long pseudorandom stream, a secure stream cipher is natively suited to act as a cryptographically secure PRNG (CSPRNG). The standard for this today is ChaCha20 [18]. It has largely replaced older ciphers because it is exceptionally fast on modern processors, often outperforming hardware-accelerated ciphers while maintaining a massive security margin.

**Operational Mechanics** The internal state of ChaCha20 is structured as a  $4 \times 4$  matrix of 32-bit words. For a standard 256-bit key and 96-bit nonce (or PRNG seed equivalent), the initial state matrix  $S$  is populated as follows:

- **Row 0 (Constants):** Four fixed ASCII words ("expand 32-byte k").
- **Rows 1–2 (Key):** The 256-bit secret key, split into eight 32-bit words.
- **Row 3 (Counter and Nonce):** A 32-bit block counter usually initialized to 0, followed by the 96-bit nonce.

Mathematically, the initial state  $S$  is visualized as:

$$S = \begin{bmatrix} \text{Const}_0 & \text{Const}_1 & \text{Const}_2 & \text{Const}_3 \\ \text{Key}_0 & \text{Key}_1 & \text{Key}_2 & \text{Key}_3 \\ \text{Key}_4 & \text{Key}_5 & \text{Key}_6 & \text{Key}_7 \\ \text{Counter} & \text{Nonce}_0 & \text{Nonce}_1 & \text{Nonce}_2 \end{bmatrix}$$

**The Quarter Round (QR) Operation** The cryptographic mixing is driven by the Quarter Round function. It is an ARX (Add-Rotate-XOR) construction, taking four 32-bit words  $(a, b, c, d)$  and updating them using addition modulo  $2^{32}$ , bitwise XOR ( $\oplus$ ), and left rotations ( $\ll$ ). A single Quarter Round executes the following sequence:

$$\begin{aligned} a &\leftarrow a + b; & d &\leftarrow (d \oplus a)16 \\ c &\leftarrow c + d; & b &\leftarrow (b \oplus c)12 \\ a &\leftarrow a + b; & d &\leftarrow (d \oplus a)8 \\ c &\leftarrow c + d; & b &\leftarrow (b \oplus c)7 \end{aligned}$$

**Block Generation and Output** To generate a 64-byte keystream block, the algorithm applies the QR operation 20 times across the state matrix:

1. **Column Rounds (Odd rounds):** The QR operation is applied vertically to each of the four columns.
2. **Diagonal Rounds (Even rounds):** The QR operation is applied diagonally to ensure full diffusion across the matrix.

After 20 rounds, the resulting scrambled matrix is added word-by-word to the original starting matrix  $S$  (modulo  $2^{32}$ ). This final addition step is critical: it acts as a one-way function, preventing an attacker who observes the output from simply running the Quarter Rounds in reverse to deduce the internal state. To produce the final 64-byte pseudorandom sequence, the 16 words are serialized using little-endian byte ordering. The counter is then incremented to generate the next block.

**Security and Statistical Performance** The cryptographic strength of ChaCha20 relies on the rigorous mathematical principles of *confusion and diffusion* achieved through its ARX (Add-Rotate-XOR) architecture. Because it mixes operations from fundamentally incompatible algebraic groups (bitwise XOR and rotations over the finite field  $F_2$  alongside modular addition over the ring  $Z_{2^{32}}$ ), the generator strongly resists algebraic cryptanalysis. Furthermore, it achieves absolute diffusion extremely rapidly: a single bit

change in the initial state propagates to affect every single word in the matrix after just 2.5 rounds.

Empirically, its output is computationally indistinguishable from true randomness, allowing it to pass standard suites like NIST SP 800-22, as well as the most punishing empirical suites available, such as TestU01's *BigCrush* and *PractRand*. Research indicates that the core permutation achieves perfect statistical randomness after only 8 rounds (ChaCha8). The full 20-round specification exists purely to provide a massive cryptographic security margin against future cryptanalytic advancements.

**Advantages** ChaCha20 is widely used for software-based CSPRNGs due to its distinct architectural design:

- **Software performance:** ChaCha20 uses only fast operations, making it ideal for mobile devices, embedded systems, and broad software applications.
- **High Parallelizability:** Because the internal state relies on a sequential block counter, any part of the keystream can be computed entirely independently. Modern processors can leverage vector instructions to compute multiple blocks of the matrix simultaneously, resulting in immense data throughput.
- **Cache-Timing immunity:** By relying exclusively on constant-time ARX operations, ChaCha20 completely avoids data-dependent memory lookups (S-boxes). This design makes the generator natively immune to cache-timing side-channel attacks.

**Limitations and Drawbacks** Despite its performance, using a stream cipher as a PRNG requires strict operational hygiene to avoid catastrophic failures:

- **State Reuse:** The absolute cardinal sin of any stream cipher is initializing it with the exact same key and nonce (or seed) twice. Doing so will generate the exact same pseudorandom sequence, entirely destroying its unpredictability. Rigorous entropy management and strict state uniqueness are mathematically critical for its secure operation.
- **Lack of native forward secrecy:** A standalone stream cipher guarantees unpredictability only as long as its internal state remains secret. If a sophisticated attacker compromises the system memory and reads the current ChaCha20 state matrix, they can predict all future outputs.

## 5 Implementation

### 5.1 CTR-DRBG based on AES

The NIST SP 800-90A[12] standard defines CTR-DRBG, a deterministic random bit generator based on a block cipher such as AES.

The construction maintains an internal state composed of a secret key  $K$  and a counter value  $V$ . At each iteration, the generator produces a block of pseudorandom output by encrypting the current counter value:

$$R_i = AES_K(V_i)$$

where  $V_i$  denotes the internal counter at step  $i$ . After each computation, the counter is incremented to ensure that a new input is used at every iteration:

$$V_{i+1} = (V_i + 1) \bmod 2^{128}$$

The sequence of outputs is obtained by concatenating successive blocks:

$$R_1 || R_2 || R_3 || \dots$$

forming a pseudorandom bitstream.

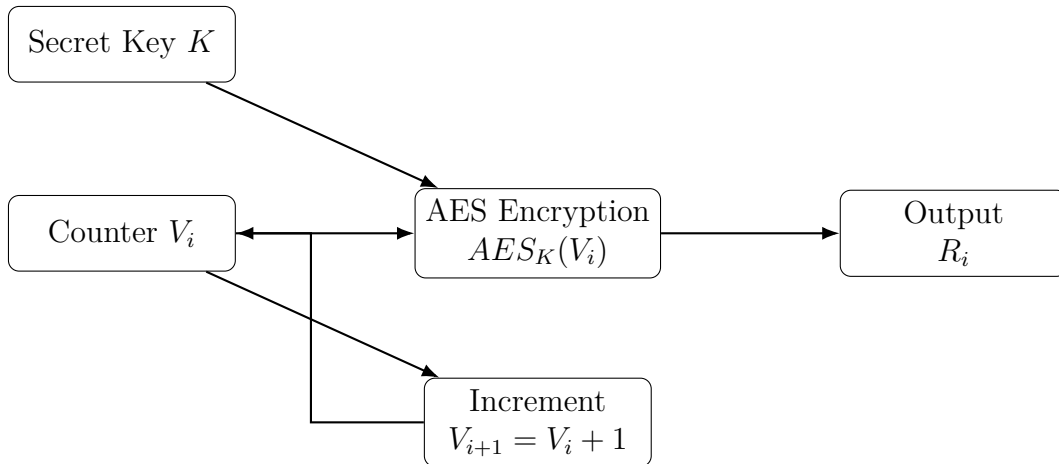


Figure 1: Simplified structure of CTR-DRBG based on AES

Intuitively, the security of CTR-DRBG comes from the fact that AES behaves like a pseudorandom permutation when the key is secret. Even if the counter values  $V_i$  are fully predictable, the encryption hides any structure and produces outputs that appear random.

The secrecy of the key  $K$  is therefore the critical element: without it, an attacker cannot predict future outputs or reconstruct previous ones. But if he gets the key and succeeds to recover the value of the counter, he can reconstruct previous outputs and guess the next ones.

Another important property is that the counter ensures all inputs  $V_i$  are distinct. This prevents repetition of inputs to AES, which would otherwise lead to repeated outputs and potential information leakage.

In practice, this means that although the generator is fully deterministic, its output cannot be distinguished from true randomness by any efficient adversary.

The same principle is used in practice for encryption in counter (CTR) mode. The pseudorandom blocks generated by encrypting successive counter values can be used as a keystream, which is then combined with the plaintext using a bitwise XOR operation. This transforms the block cipher into a stream cipher: the ciphertext is obtained as

$$C_i = M_i \oplus R_i,$$

and decryption is performed identically. This approach highlights the close connection between pseudorandom generation and symmetric encryption.

## 5.2 Implementation Architecture

Our implementation of the CTR\_DRBG based on AES-256 is divided into two primary files to ensure a clean separation between the cryptographic algorithm and the interaction with users and hardware: `ctr_drbg.c` and `main.c`.

### 5.2.1 Code organization and entropy sourcing

The `main.c` file serves as the entry point and the consuming application. It is responsible for handling user inputs, managing dynamic memory allocation for the requested output, and orchestrating the lifecycle of the DRBG (initialization, generation, handling reseed requests, and secure uninstantiation).

To initialize and reseed the DRBG, the application must provide cryptographic-quality randomness. In our implementation, we source this entropy directly from the Unix-like operating system's `/dev/urandom` special file. `/dev/urandom` provides a continuous stream of cryptographically secure pseudorandom numbers derived from the kernel's entropy pool. By reading directly from this device, we guarantee that the initial seed material possesses the full entropy required to satisfy the security claims of the AES-256 generator.

### 5.2.2 The derivation function

According to the NIST SP 800-90A Rev. 1 standard, a CTR\_DRBG can be implemented either with or without a Block Cipher Derivation Function. The Derivation Function is designed to compress and mix entropy sequences of varying lengths or lower-quality entropy sources into a uniformly distributed seed.

We chose to implement the CTR\_DRBG without a Derivation Function. This design choice imposes a strict operational requirement: the entropy input must exactly match the *seedlen* (which is 48 bytes, corresponding to the concatenation of a 256-bit key and a 128-bit counter) and must consist of full entropy. Furthermore, the standard dictates that a separate *nonce* is not used in this mode. Our application satisfies this requirement by pulling exactly 48 bytes of high-quality entropy from `/dev/urandom` during both the instantiation and reseeding phases.

### 5.2.3 Data structure and security bounds

To maintain the internal memory of the generator across multiple function calls, the implementation relies on a specific state context structure, `CTR_DRBG_CTX`. According to the NIST standard, this internal state is composed of three critical components: the cryptographic Key, the state variable `V`, and the `reseed_counter`.



The generator operates using a dual-counter system, where `V` and the `reseed_counter` serve different cryptographic purposes:

**The cryptographic state (`V` and `Key`):** `V` is the 128-bit (16-byte) block counter required by the AES block cipher operating in Counter mode (CTR). During a generation request, `V` is incremented for *every single 16-byte block* of output produced. AES encrypts `V` using the 256-bit `Key` to produce the pseudorandom bitstream. Thus, `V` is the core moving part of the cryptographic engine.

**The `reseed_counter`:** Unlike `V` which increments per block, the `reseed_counter` increments only once per *generation request* (regardless of whether the application asked for 1 byte or 10,000 bytes). Its purpose is to track the lifespan of the current entropy seed. Generating too many outputs from a single initial seed could theoretically expose the generator to cryptanalytic attacks. By enforcing a strict reseed interval, the generator ensures that the internal state is periodically refreshed. When the threshold is reached, the DRBG automatically triggers a reseed operation internally within the generation function, using fresh entropy from the operating system.

**Maximum Output Length per Request** In addition to the reseed counter, our implementation enforces a strict maximum output length per generation request, capped at  $2^{19}$  bits (65,536 bytes) as demanded by the NIST standard. This constraint follows the security recommendations of the NIST SP 800-90A standard.

This limitation bounds the amount of data generated under a single internal state (`Key` and `V`). Restricting the output size reduces the exposure of the generator before the internal state is refreshed through the `ctr_drbg_update` mechanism.

After each generation request, the generator performs a state update which derives a fresh `Key` and counter value `V`. This mechanism provides *backtracking resistance*: even if the current internal state is later compromised, previously generated outputs cannot be reconstructed efficiently.

#### 5.2.4 Core functions in `ctr_drbg.c`

The `ctr_drbg.c` file encapsulates the internal state and the strict algorithmic flow of the generator. It implements the following fundamental functions:

**`increment_V`** A static utility function that increments the 128-bit internal counter `V`. It processes the byte array in big-endian format, ensuring that the AES block cipher operates on a strictly monotonic sequence to prevent cycles.

**`ctr_drbg_update`** It encrypts the incremented counter `V` three times (generating 48 bytes for AES-256) to produce a new temporary key and counter. If optional *provided\_data* (such as a personalization string or additional input) is given, it is securely XORed into the state. Finally, the internal `Key` and `V` are overwritten with the new values.

**`ctr_drbg_init`** Initializes the DRBG context. It zeroizes the internal state, concatenates the 48-byte full-entropy input (optionally XORed with a personalization string), and passes it to the `update` function to securely derive the initial `Key` and `V`. It also sets the reseed counter to 1.

**ctr\_drbg\_generate** The primary function used by the consuming application to obtain pseudorandom bits. It verifies that the maximum output length per request (65,536 bytes) is not exceeded and checks if the generator requires reseeding. It then enters a loop, incrementing  $V$  and encrypting it with AES-256 until the requested number of bytes is produced. To ensure *backtracking resistance*, the state is immediately updated again at the end of the generation cycle.

**ctr\_drbg\_reseed** Restores the security strength of the generator when the reseed counter limit is reached. It takes 48 bytes of fresh entropy from the OS, optionally mixes it with additional input, and updates the internal state, resetting the reseed counter back to 1. Furthermore, reseeding also contributes to *prediction resistance*: after fresh entropy is injected into the internal state, previous observations of the generator output no longer help predicting future outputs.

**ctr\_drbg\_uninstantiate** A critical security function that securely zeroes out the internal state (Key,  $V$ , and counters) when the DRBG is no longer needed, preventing residual secrets from leaking into the system’s memory.

## 5.3 Testing and Validation

To validate the correctness and statistical properties of the implemented CTR\_DRBG, we designed a multi-layered testing strategy combining functional verification, compliance checks with NIST standards, and empirical randomness evaluation.

### 5.3.1 Functional and Compliance Testing

The functional correctness of the generator is verified with respect to the requirements defined in the NIST SP 800-90A Rev. 1 standard, which specifies the behavior of deterministic random bit generators (DRBGs).

In particular, the following properties are validated:

- Correct instantiation, generation, reseeding, and uninstantiation procedures.
- Proper management of internal state variables (Key,  $V$ , and `reseed_counter`).
- Enforcement of the maximum number of bits per request ( $2^{19}$  bits / 65,536 bytes), as required by the standard.
- Correct triggering of the reseed mechanism after reaching the reseed interval.

### 5.3.2 Statistical Randomness Testing

To assess the statistical quality of the generated output, we apply a subset of the tests defined in the NIST SP 800-22 statistical test suite [15].

The following tests are implemented:

- Frequency (Monobit) Test
- Runs Test
- Block Frequency Test

- Approximate Entropy Test
- Cumulative Sums Test
- Linear Complexity Test
- Discrete Fourier Transform (Spectral) Test

These tests evaluate different statistical properties of the output sequence, including balance between zeros and ones, distribution of patterns, independence between bits, and absence of periodic structures.

The implementation computes p-values using normal and chi-square approximations, and a sequence is considered statistically acceptable if the p-value is greater than or equal to 0.01, following the common NIST significance threshold.

### 5.3.3 Additional Empirical Tests

In addition to the NIST-based evaluation, supplementary tests were implemented to further characterize the generator:

- Autocorrelation test to measure dependence between adjacent bits.
- Performance comparison against Linux entropy sources (`getrandom` and `/dev/urandom`).
- Chi-square and uniformity analysis at byte level.

These tests are not part of the official NIST suite but provide additional empirical evidence regarding the statistical independence and efficiency of the implementation.

### 5.3.4 Tests result & Interpretation

```
marilou@marilou-IdeaPad-5-15ALC05:~/Documents/CTR-DRBG/aes_ctr_drbg_project$ ./test/run_tests
STARTING CTR_DRBG TEST SUITE

--- Running state tests (Uninstantiate) ---
[SUCCESS] ctr_drbg_uninstantiate écrase correctement Key, V et reseed_counter avec des zéros

Running NIST constraint tests
[SUCCESS] La génération échoue si on demande > 65536 octets d'un coup (Max_Bits_Per_Request)
[SUCCESS] La génération réussit lorsque le compteur est exactement à la limite (RESEED_INTERVAL)
[SUCCESS] Le générateur se verrouille et exige un reseed quand l'intervalle est dépassé
[SUCCESS] La fonction reseed réinitialise correctement le compteur à 1
[SUCCESS] Le générateur est correctement débloqué et génère des octets après le reseed

Running Known Answer Test (KAT)
[SUCCESS] Snapshot du KAT enregistré (vecteur déterministe)

--- Frequency (Monobit) Test ---
S_n      = -110
s_obs    = 0.245967
p-value  = 0.805707
[SUCCESS] NIST Frequency (Monobit) Test

--- Test runs (séquences 0/1) ---
pi       = 0.498456
Vn (runs) = 80241
Z statistic = 1.208825
p-value  = 0.226730
[SUCCESS] Runs Test

--- Autocorrelation Test ---
[SUCCESS] Autocorrelation Test
```

Figure 2: Tests result (first part)

```

--- Test Block Frequency ---
[SUCCESS] Block Frequency Test

--- Approximate Entropy Test ---
[SUCCESS] Approximate Entropy Test (NIST)

--- Cumulative Sums Test) ---
[SUCCESS] Cumulative Sums Test

--- Linear Complexity Test ---
[SUCCESS] Linear Complexity Test

--- Spectral Test ---
[SUCCESS] Spectral Test

Running Known Answer Test (KAT)
[SUCCESS] Le résultat déterministe (KAT) correspond parfaitement

                        FINAL REPORT
Tests executed      : 16
Tests passed       : 16 (100%)

>>> ALL TESTS PASSED SUCCESSFULLY! <<<

--- Comparison CTR_DRBG vs Linux RNG ---

--- SPEED ---
CTR_DRBG           : 0.0000 s
getrandom()        : 0.0031 s
/dev/urandom       : 0.0031 s

--- CHI-SQUARE ---
CTR_DRBG           : 255000000.00
getrandom()        : 252.03
/dev/urandom       : 270.17

--- UNIFORMITY (max deviation) ---
CTR_DRBG           : 0.996094
getrandom()        : 0.000160
/dev/urandom       : 0.000187

```

Figure 3: Tests result (second part)

We can see that all our tests are working correctly. Note that this is not the case every time we run the program for some statistical tests, which is normal according to the NIST SP 800-22 statistical test suite [15], since these tests are probabilistic in nature and may occasionally produce false positives or false negatives even for a correct random number generator.

The comparison with native Linux entropy sources highlights both performance and statistical differences.

As expected, the CTR\_DRBG implementation is significantly faster than kernel-based entropy sources such as `getrandom()` and `/dev/urandom`, since it is purely deterministic and does not depend on hardware entropy collection at generation time.

However, the statistical indicators (chi-square and uniformity deviation) show a clear discrepancy between CTR\_DRBG and Linux RNG outputs. The very high chi-square value observed for CTR\_DRBG suggests that raw byte-level statistical measures are not directly comparable between deterministic generators and kernel-based entropy sources when used in isolation. This limitation is due to the structured and stateful nature of CTR\_DRBG.

In contrast, `getrandom()` and `/dev/urandom` exhibit significantly lower chi-square values and near-zero deviation from uniformity, which is consistent with well-mixed kernel entropy outputs.

Overall, these results confirm that while CTR\_DRBG is efficient and deterministic by design, statistical evaluation must be interpreted carefully, as raw chi-square comparisons alone are insufficient to assess cryptographic quality without considering the underlying generator model.

The implemented CTR\_DRBG exhibits linear computational complexity with respect to the number of generated bytes, as each 16-byte block requires a single AES-256 encryption operation. This results in an overall time complexity of  $O(n)$ , where  $n$  is the output size. In contrast, the memory complexity remains constant, since the internal state is fixed in size (key, counter, and control variables) and does not grow with the amount of generated data. These properties make the generator both efficient and predictable in performance, which is consistent with the design of block-cipher-based DRBGs defined in NIST SP 800-90A.

**Comments:** *We have included some implementation files in the appendix.*

## 6 Conclusion

PRGs play a central role in modern cryptography, providing the randomness required by secure protocols. As this report has shown, the notion of randomness in cryptography goes far beyond simple statistical properties: a generator must satisfy strong security requirements such as computational indistinguishability, unpredictability, and resistance to state compromise.

We have seen that classical generators based on linear recurrences, although efficient and statistically well-behaved, are fundamentally insecure due to their algebraic structure, which allows efficient state recovery attacks. This highlights a crucial lesson: good statistical properties alone are not sufficient for cryptographic security.

In contrast, modern cryptographically secure generators rely on well-established primitives such as hash functions, block ciphers, or stream ciphers, whose security has been extensively studied. Standardized constructions such as DRBGs combine these primitives with careful state management and reseedling mechanisms to achieve both efficiency and strong security guarantees.

We also examined how randomness is handled in practice, where physical entropy sources are used to initialize deterministic generators. This hybrid approach ensures both high performance and strong unpredictability, and is now the standard design in modern operating systems and cryptographic libraries.

Finally, the study of a concrete implementation, CTR-DRBG based on AES, illustrates how theoretical concepts are applied in real-world systems, bridging the gap between abstract definitions and practical deployment.

To synthesize the operational and security characteristics of the various constructions analyzed throughout this report, Table 1 summarizes their respective advantages, structural vulnerabilities, and primary use cases.

Overall, pseudorandom generation is a subtle and critical component of cryptographic security. Its design requires a careful balance between theory and practice, and mistakes in this area can have severe consequences. Looking forward, as cryptographic standards evolve to resist quantum computing, the demand for high-throughput, rigorously proven randomness will only increase, cementing the DRBG as an indispensable tool for future security architectures.

| Generator                                    | Advantages                                                                               | Inconveniences                                                                                                      | Practical Usage                                                                         |
|----------------------------------------------|------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <b>Linear Congruential (LCG)</b>             | fast, simple to implement, requires very little memory.                                  | Zero cryptographic security. Linear structure allows full state recovery via simple algebra or LLL lattice attacks. | <b>Non-Cryptographic.</b> Obsolete for security, relegated to statistical simulation.   |
| <b>Permuted Congruential (PCG)</b>           | Combines LCG speed with a permutation step for better statistical distribution.          | Not cryptographically secure. Vulnerable to lattice-based guess-and-determine attacks.                              | <b>Non-Cryptographic.</b> Popular in scientific simulation (e.g., NumPy).               |
| <b>Linear Feedback Shift Register (LFSR)</b> | fast, hardware-efficient, excellent basic statistical properties.                        | Highly predictable due to linear structure. State easily recovered via Berlekamp-Massey algorithm.                  | <b>Non-Cryptographic.</b> Used in communication systems but never as standalone crypto. |
| <b>Mersenne Twister</b>                      | Exceptional software speed, massive period, excellent multidimensional equidistribution. | Entirely linear over $F_2$ . Observing 624 outputs completely reveals the internal state.                           | <b>Non-Cryptographic.</b> Standard in programming environments for simulation only.     |
| <b>Blum-Blum-Shub (BBS)</b>                  | Strong, provable theoretical security                                                    | Too slow. Requires modular squaring of large integers to extract very few bits.                                     | <b>Theoretical.</b> Rarely used in real-world systems.                                  |
| <b>Hash-DRBG</b>                             | Conceptually simple, highly portable, provides absolute backtracking resistance.         | Historically slower than block ciphers in software without dedicated hardware instructions.                         | <b>Cryptographic.</b> Widely deployed in security modules and modern operating systems. |
| <b>Block-Cipher (CTR-DRBG)</b>               | High security, benefits from hardware acceleration (e.g., AES-NI).                       | Security relies entirely on strict key secrecy                                                                      | <b>Cryptographic.</b> A modern standardized design for secure bit generation.           |
| <b>Stream-Cipher (ChaCha20)</b>              | fast, highly parallelizable, immune to cache-timing attacks                              | Catastrophic failure if state/nonce is reused; lacks native forward secrecy without proper reseeding.               | <b>Cryptographic.</b> The modern standard for software-based CSPRNGs.                   |

Table 1: Summary of PRGs: Characteristics and Usage



## References

- [1] O. Goldreich, *Foundations of Cryptography: A Primer*, Now Publishers, 2008.
- [2] J. Katz, Y. Lindell, *Introduction to Modern Cryptography*, 3rd edition, CRC Press, 2020.
- [3] A. C.-C. Yao, "Theory and Applications of Trapdoor Functions", FOCS, 1982.
- [4] A. Menezes, P. van Oorschot, S. Vanstone, *Handbook of Applied Cryptography*, CRC Press, 1996.
- [5] L. Blum, M. Blum, M. Shub, "A Simple Unpredictable Pseudo-Random Number Generator", *SIAM Journal on Computing*, 1986.
- [6] M. Matsumoto, T. Nishimura, "Mersenne Twister: A 623-dimensionally equidistributed uniform pseudorandom number generator", *ACM Transactions on Modeling and Computer Simulation*, 1998.
- [7] S. W. Golomb, *Shift Register Sequences*, Aegean Park Press, 1982.
- [8] M. E. O'Neill, "PCG: A Family of Simple Fast Space-Efficient Statistically Good Algorithms for Random Number Generation", 2014.
- [9] C. Bouillaguet, F. Martinez, J. Sauvage, "Practical seed-recovery for the PCG pseudo-random number generator", *IACR Transactions on Symmetric Cryptology*, Vol. 2020, No. 3, pp. 175–196, 2020.
- [10] ANSI, *ANSI X9.31: Digital Signatures Using Reversible Public Key Cryptography for the Financial Services Industry*, 1998.
- [11] B. Kelsey, J. Schneier, D. Wagner, C. Hall, "Cryptanalytic Attacks on Pseudorandom Number Generators", *Fast Software Encryption (FSE)*, 1998.
- [12] NIST, *SP 800-90A Rev.1: Recommendation for Random Number Generation Using Deterministic Random Bit Generators*, 2015.
- [13] NIST, *SP 800-90B: Recommendation for the Entropy Sources Used for Random Bit Generation*, 2018.
- [14] NIST, *SP 800-90C: Recommendation for Random Bit Generator Constructions*, 2016.
- [15] NIST, *SP 800-22 Rev.1a: Statistical Test Suite for Random and Pseudorandom Number Generators*, 2010.
- [16] P. L'Ecuyer, R. Simard, "TestU01: A C Library for Empirical Testing of Random Number Generators", *ACM Transactions on Mathematical Software*, 2007.
- [17] D. Eastlake et al., *RFC 4086: Randomness Requirements for Security*, IETF, 2005.
- [18] Y. Nir, A. Langley, *RFC 8439: ChaCha20 and Poly1305 for IETF Protocols*, IETF, 2018.

# A Appendices

## A.1 CTR\_DRBG Implementation

### A.1.1 Header File: ctr\_drbg.h

```
#ifndef CTR_DRBG_H
#define CTR_DRBG_H
#define RESEED_INTERVAL 10000
#include <stdint.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include "aes.h"
#include "key_expansion.h"

#define CTR_DRBG_ENTROPY_LEN 48

typedef struct {
    uint8_t Key[32];
    uint8_t V[16];
    uint64_t reseed_counter;
} CTR_DRBG_CTX;

void ctr_drbg_init(
    CTR_DRBG_CTX *ctx,
    const uint8_t *entropy, // Must be 48 bytes
    const uint8_t *personalization_string // Optional, up to 48 bytes
);

int ctr_drbg_generate(
    CTR_DRBG_CTX *ctx,
    uint8_t *output,
    size_t outlen,
    const uint8_t *additional_input // Optional, up to 48 bytes
);

void ctr_drbg_reseed(
    CTR_DRBG_CTX *ctx,
    const uint8_t *new_entropy, // Must be 48 bytes
    const uint8_t *additional_input // Optional, up to 48 bytes
);

void ctr_drbg_uninstantiate(CTR_DRBG_CTX *ctx);

#endif
```

### A.1.2 Source File: ctr\_drbg.c

```
#include "ctr_drbg.h"

// Incrémentation du compteur V (big-endian)
static void increment_V(uint8_t V[16]) {
    for (int i = 15; i >= 0; i--) {
        V[i]++;
        if (V[i] != 0) {
            break;
        }
    }
}

// mise à jour de la clé et du vecteur V en utilisant la clé actuelle
static void ctr_drbg_update(CTR_DRBG_CTX *ctx, const uint8_t *provided_data) {
    uint8_t temp[48]; // 32 octets (Key) + 16 octets (V) pour AES-256
    uint8_t block[16];
    AESRoundKeys rk;

    // On étend la clé ACTUELLE pour chiffrer
    KeyExpansion(ctx->Key, rk);
```

```

// Générer 48 octets en chiffrant les compteurs successifs
for (int i = 0; i < 3; i++) {
    increment_V(ctx->V);
    AES_encrypt_block(ctx->V, block, rk); // V = V + 1 // Chiffrer V
    memcpy(temp + (i * 16), block, 16); // Stocker le bloc dans temp
}

// XOR avec les provided_data
if (provided_data != NULL) {
    for (int i = 0; i < 48; i++) {
        temp[i] ^= provided_data[i];
    }
}

// Mettre à jour le contexte avec les nouvelles valeurs
memcpy(ctx->Key, temp, 32); // Les 32 premiers octets = Nouvelle Clé
memcpy(ctx->V, temp + 32, 16); // Les 16 suivants = Nouveau Vecteur V

// Nettoyage de sécurité (effacer les traces en mémoire)
memset(temp, 0, sizeof(temp));
memset(block, 0, sizeof(block));
memset(rk, 0, sizeof(rk));
}

// Initialisation
void ctr_drbg_init(CTR_DRBG_CTX *ctx, const uint8_t *entropy, const uint8_t *perso_string) {
    //on met tout à zéro
    memset(ctx->Key, 0, 32);
    memset(ctx->V, 0, 16);
    ctx->reseed_counter = 1;

    uint8_t seed_material[48];
    memcpy(seed_material, entropy, 48); // Entropy MUST be 48 bytes

    // XOR personalization string into seed_material if provided
    if (perso_string != NULL) {
        for (int i = 0; i < 48; i++) {
            seed_material[i] ^= perso_string[i];
        }
    }
    //on utilise update pour utilis l'entropie
    ctr_drbg_update(ctx, seed_material);
    memset(seed_material, 0, sizeof(seed_material));
}

// Génération
int ctr_drbg_generate(CTR_DRBG_CTX *ctx, uint8_t *output, size_t outlen, const uint8_t *additional_input)
{ //on vérifie les conditions
    if (outlen > 65536) { // NIST Max limit (2^19 bits)
        return -1;
    }

    if (ctx->reseed_counter > RESEED_INTERVAL) {
        fprintf(stderr, "Erreur : Reseed requis !\n");
        return -1;
    }

    if (additional_input != NULL) {
        ctr_drbg_update(ctx, additional_input);
    }

    AESRoundKeys rk;
    KeyExpansion(ctx->Key, rk);

    size_t produced = 0;
    //boucle de génération : on incrémente V, on chiffre, on copie dans output
    while (produced < outlen) {
        increment_V(ctx->V);
        uint8_t block[16];
        AES_encrypt_block(ctx->V, block, rk);

        size_t n = (outlen - produced > 16) ? 16 : (outlen - produced);
        memcpy(output + produced, block, n);
        produced += n;
    }
}

```

```

        memset(block, 0, sizeof(block));
    }

    ctr_drbg_update(ctx, additional_input); // Update again to prevent backtracking
    ctx->reseed_counter++;

    memset(rk, 0, sizeof(rk));
    return 0;
}

// Mise à jour de l'état avec une nouvelle entropie (reseed)
void ctr_drbg_reseed(CTR_DRBG_CTX *ctx, const uint8_t *new_entropy, const uint8_t *additional_input) {
    uint8_t seed_material[48];
    memcpy(seed_material, new_entropy, 48);

    if (additional_input != NULL) {
        for (int i = 0; i < 48; i++) {
            seed_material[i] ^= additional_input[i];
        }
    }

    ctr_drbg_update(ctx, seed_material);
    ctx->reseed_counter = 1;
    memset(seed_material, 0, sizeof(seed_material));
}

//libère la mémoire proprement
void ctr_drbg_uninstantiate(CTR_DRBG_CTX *ctx) {
    memset(ctx->Key, 0, 32);
    memset(ctx->V, 0, 16);
    ctx->reseed_counter = 0;
}

```

### A.1.3 Main Program: main.c

```

#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h> // Pour open()
#include <unistd.h> // Pour read() et close()
#include "ctr_drbg.h"

void get_randomness(uint8_t *buffer, size_t length) {
    int fd = open("/dev/urandom", O_RDONLY);
    if (fd < 0) {
        perror("Erreur : impossible d'ouvrir /dev/urandom");
        exit(EXIT_FAILURE);
    }

    ssize_t result = read(fd, buffer, length);
    if (result < 0 || (size_t)result != length) {
        perror("Erreur lors de la lecture de l'entropie");
        close(fd);
        exit(EXIT_FAILURE);
    }
    close(fd);
}

int main(int argc, char **argv) {

    // Définir une longueur par défaut (ex: 64)
    size_t length = 64;

    // Si l'utilisateur passe un argument, on l'utilise comme longueur
    if (argc > 1) {
        int input_length = atoi(argv[1]);
        if (input_length > 0) {
            length = (size_t)input_length;
        } else {
            printf("Erreur : la longueur doit être un entier positif.\n");
            return 1;
        }
    }
}

```

```

CTR_DRBG_CTX ctx;

// il faut 48 octets d'entropie pure pour AES-256
uint8_t entropy[48];

// Allouer dynamiquement le tableau en fonction de la taille demandée
uint8_t *output = (uint8_t *)malloc(length * sizeof(uint8_t));
if (output == NULL) {
    printf("Erreur d'allocation mémoire.\n");
    return 1;
}

// Remplir le tableau avec du hasard physique (48 octets)
get_randomness(entropy, sizeof(entropy));

// Initialisation (pas de nonce, on passe NULL pour la personalization string)
ctr_drbg_init(&ctx, entropy, NULL);

// Générer le nombre d'octets demandés (on passe NULL pour l'additional_input)
int status = ctr_drbg_generate(&ctx, output, length, NULL);

// Si le générateur est bloqué
if (status != 0) {
    printf("Le générateur a atteint sa limite. Réamorçage en cours...\n");

    // 1. On va chercher de la nouvelle entropie physique (48 octets obligatoires)
    uint8_t new_entropy[48];
    get_randomness(new_entropy, sizeof(new_entropy));

    // 2. On débloquent le générateur avec la fonction reseed (NULL pour l'additional_input)
    ctr_drbg_reseed(&ctx, new_entropy, NULL);

    // 3. On relance la génération qui avait échoué
    ctr_drbg_generate(&ctx, output, length, NULL);
}

// Afficher le résultat
for (size_t i = 0; i < length; i++) {
    // On ajoute un espace après le %02x
    printf("%02x ", output[i]);

    if ((i + 1) % 16 == 0)
        printf("\n");
}

// Retour à la ligne final si la taille n'est pas un multiple de 16
if (length % 16 != 0) {
    printf("\n");
}

// Effacement sécurisé du contexte DRBG de la mémoire
ctr_drbg_uninstantiate(&ctx);

// Libérer la mémoire allouée
free(output);

return 0;
}

```

## A.2 Implementation of CTR\_DRBG Tests

### A.2.1 Tests File: tests.c

```

#define _POSIX_C_SOURCE 200809L // pour fileno()
#include <stdio.h>
#include <unistd.h> // pour dup, dup2, close
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <math.h>
#include <complex.h>
#include <time.h>
#include <sys/random.h>

```

```

#include "ctr_drbg.h"

// Définition de M_PI si nécessaire
#ifndef M_PI
#define M_PI 3.14159265358979323846
#endif

//pour compiler faire make test
//pour exécuter faire ./test/run_tests

// Variables globales pour le rapport de test
int tests_run = 0;
int tests_passed = 0;
int tests_failed = 0;

// Fonction utilitaire pour évaluer un test
void assert_test(int condition, const char *test_name) {
    tests_run++;
    if (condition) {
        printf("[SUCCESS] " " %s\n", test_name);
        tests_passed++;
    } else {
        printf( "[FAILURE] " " %s\n", test_name);
        tests_failed++;
    }
}

// Convertit une chaîne hexadécimale (ex: "A1B2C3...") en tableau d'octets
void hex_to_bytes(const char *hex_string, uint8_t *byte_array, size_t array_len) {
    for (size_t i = 0; i < array_len; i++) {
        sscanf(hex_string + 2 * i, "%2hhx", &byte_array[i]);
    }
}

// 1. Tests d'instanciation et destruction
void test_uninstantiate() {
    printf( "\n--- Running state tests (Uninstantiate) ---\n" );

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    memset(entropy, 0xAA, 48); // Entropie factice

    ctr_drbg_init(&ctx, entropy, NULL);
    ctr_drbg_uninstantiate(&ctx);

    // Vérifier que tout est bien à zéro
    int is_zero = 1;
    for (int i = 0; i < 32; i++) if (ctx.Key[i] != 0) is_zero = 0;
    for (int i = 0; i < 16; i++) if (ctx.V[i] != 0) is_zero = 0;
    if (ctx.reseed_counter != 0) is_zero = 0;

    assert_test(is_zero, "ctr_drbg_uninstantiate écrase correctement Key, V et reseed_counter avec des
    ↪ zéros");
}

// 2. Tests des limites imposées par le NIST
void test_constraints() {
    printf( "\nRunning NIST constraint tests\n" );

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    memset(entropy, 0xBB, 48);

    ctr_drbg_init(&ctx, entropy, NULL);

    // Test A : Limite de taille par requête (Max 65536 octets)
    uint8_t dummy_out[1];
    int status = ctr_drbg_generate(&ctx, dummy_out, 65537, NULL);
    assert_test(status == -1, "La génération échoue si on demande > 65536 octets d'un coup
    ↪ (Max_Bits_Per_Request)");

    // Test B : Intervalle de Reseed
    // On simule artificiellement que le générateur a beaucoup tourné
    ctx.reseed_counter = RESEED_INTERVAL;

```

```

// AJOUT : redirection de stderr pour masquer le message
int old_stderr_fd = dup(fileno(stderr));
freopen("/dev/null", "w", stderr); // redirige stderr vers /dev/null

status = ctr_drbg_generate(&ctx, dummy_out, 16, NULL);
assert_test(status == 0, "La génération réussit lorsque le compteur est exactement à la limite
↳ (RESEED_INTERVAL)");

status = ctr_drbg_generate(&ctx, dummy_out, 16, NULL);
assert_test(status == -1, "Le générateur se verrouille et exige un reseed quand l'intervalle est
↳ dépassé");

fflush(stderr);
dup2(old_stderr_fd, fileno(stderr)); // restaure stderr
close(old_stderr_fd);
// FIN de la redirection

// Test C : Reseed et déblocage
uint8_t new_entropy[48];
memset(new_entropy, 0xCC, 48);
ctr_drbg_reseed(&ctx, new_entropy, NULL);

assert_test(ctx.reseed_counter == 1, "La fonction reseed réinitialise correctement le compteur à 1");

status = ctr_drbg_generate(&ctx, dummy_out, 16, NULL);
assert_test(status == 0, "Le générateur est correctement débloquent et génère des octets après le
↳ reseed");

ctr_drbg_uninstantiate(&ctx);
}

// 3. Known Answer Test (KAT)
void test_known_answers() {
    printf( "\nRunning Known Answer Test (KAT)\n" );

    // Note : Pour valider officiellement l'algorithme, ces vecteurs doivent être extraits
    // des fichiers officiels CAVP (Cryptographic Algorithm Validation Program) du NIST.
    // Ceci est un squelette de test déterministe pour prouver que l'algo ne change pas.

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    uint8_t output[16];
    //uint8_t expected_output[16]; / <-- mis en commentaire car sinon Warning (car pas utilisé)

    // Remplissage prédictible (0x00, 0x01, 0x02...)
    for (int i = 0; i < 48; i++) entropy[i] = i;

    ctr_drbg_init(&ctx, entropy, NULL);
    ctr_drbg_generate(&ctx, output, 16, NULL);

    // On stocke le résultat du premier run (remplacez par un vrai vecteur NIST plus tard)
    // Ici, c'est un test de non-régression simple. S'il échoue à l'avenir, vous saurez
    // que vous avez "cassé" l'algorithme.
    static int first_run = 1;
    static uint8_t kat_snapshot[16];

    if (first_run) {
        memcpy(kat_snapshot, output, 16);
        first_run = 0;
        assert_test(1, "Snapshot du KAT enregistré (vecteur déterministe)");
    } else {
        int match = (memcmp(output, kat_snapshot, 16) == 0);
        assert_test(match, "Le résultat déterministe (KAT) correspond parfaitement");
    }

    ctr_drbg_uninstantiate(&ctx);
}

// Ici une vraie entropie aléatoire est utilisée pour que les tests statistiques
// (frequency, autocorrelation, cumulative sums) passent correctement.
// erfc() = outil pour calculer des p-values (lorsque la statistique suit une loi normale)

```

```

// NIST SP 800-22
void test_frequency() {
    printf("\n--- Frequency (Monobit) Test ---\n");

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];

    FILE *f = fopen("/dev/urandom", "rb");
    fread(entropy, 1, 48, f);
    fclose(f);

    ctr_drbg_init(&ctx, entropy, NULL);

    const int N = 200000;
    uint8_t out[N/8];

    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    int S = 0;
    //  $X_i = +1$  si bit=1,  $X_i = -1$  si bit=0
    for (int i = 0; i < N; i++) {
        int byte_index = i / 8; // octet contenant le bit i
        int bit_index = i % 8; // position du bit dans l'octet
        int bit = (out[byte_index] >> bit_index) & 1; // extraction du bit numéro i
        if (bit == 1)
            S += 1;
        else
            S -= 1;
    }

    double sobs = abs(S) / sqrt((double)N); //  $|S_n|/\sqrt{n}$  normalisation de  $S_n$ 
    //  $s_{obs}$  tend en loi vers une  $N(0,1)$  (on peut appliquer le TCL)
    double p_value = erfc(sobs / sqrt(2.0)); // p-value officielle NIST

    printf("S_n      = %d\n", S);
    printf("s_obs     = %f\n", sobs);
    printf("p-value    = %f\n", p_value);

    assert_test(p_value >= 0.01, "NIST Frequency (Monobit) Test");

    ctr_drbg_uninstantiate(&ctx);
}
// Vérifie que les 0 et 1 sont approximativement équilibrés.
// Principe : on compte tous les bits générés, puis on calcule le ratio de 1.
// Si ce ratio est proche de 0.5 (par exemple entre 0.45 et 0.55),
// la séquence est considérée "équilibrée".

// NIST SP 800-22
void test_runs() { // test des run (un run = une suite consécutive de mêmes bits)
    printf("\n--- Test runs (séquences 0/1) ---\n");

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    uint8_t out[20000];

    memset(entropy, 0xBB, 48);
    ctr_drbg_init(&ctx, entropy, NULL);

    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    const size_t n = sizeof(out) * 8;

    // calcul de la proportion de 1 ( $\pi$ )
    int ones = 0;
    for (size_t i = 0; i < n; i++) {
        int byte_index = i / 8; // octet contient le i-ème bit
        int bit_index = i % 8; // position du i-ème bit dans cet octet
        int bit = (out[byte_index] >> bit_index) & 1; // extraction
        if (bit == 1)
            ones++;
    }
    double pi = (double)ones/n; // proportion de 1

```



```

double tau = 2.0/sqrt((double)n); // Condition officielle du NIST :  $|\pi - 0.5| < 2/\sqrt{n}$ 

if (fabs(pi - 0.5) >= tau) {
    assert_test(0, "Runs Test abandonné : Frequency Test préalable échoué");
    ctr_drbg_uninstantiate(&ctx);
    return;
}

// comptage des runs :
int Vn = 1;
int prev = out[0] & 1; // prev = bit précédent dans la séquence
for (size_t i = 1; i < n; i++) {
    int Xi = (out[i/8] >> (i%8)) & 1; // extraction du bit comme avant (codé en 1 ligne)
    if (Xi != prev){
        Vn++;
    }
    prev = Xi;
}

double numerator = fabs(Vn - 2.0*n*pi*(1.0 - pi));
double denominator = 2.0*sqrt(n)*pi*(1.0 - pi);
double Z = numerator/denominator;

double p_value = erfc(fabs(Z)/sqrt(2.0)); // P_value = erfc(|Z|/sqrt(2))

printf("pi          = %f\n", pi);
printf("Vn (runs)     = %d\n", Vn);
printf("Z statistic    = %f\n", Z);
printf("p-value        = %f\n", p_value);

assert_test(p_value >= 0.01, "Runs Test"); // seuil de 1% (choisi par le NIST)

ctr_drbg_uninstantiate(&ctx);
}

// Vérifie la longueur des séquences consécutives de 0 ou 1 (runs).
// Principe : on parcourt la séquence bit par bit et on compte quand un bit change.
// Ensuite, on compare le nombre total de runs à ce qu'on attend pour un bit aléatoire (~N/2).
// Permet de détecter si les bits se regroupent trop ou pas assez.

// ce test ne fait pas partie des tests NIST, mais montre la non-corrélation des bits
// c'est un test complémentaire
void test_autocorrelation() {

    printf("\n--- Autocorrelation Test ---\n");

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    uint8_t out[200000 / 8];

    FILE *f = fopen("/dev/urandom", "rb");
    fread(entropy, 1, 48, f);
    fclose(f);

    ctr_drbg_init(&ctx, entropy, NULL);
    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    const size_t N = 200000;
    int matches = 0;

    for (size_t i = 0; i < N - 1; i++) {
        int bit_i = (out[i / 8] >> (i % 8)) & 1;
        int bit_next = (out[(i + 1) / 8] >> ((i + 1) % 8)) & 1;

        if (bit_i == bit_next)
            matches++;
    }

    double p = (double)matches / (N - 1);

    double z = (p - 0.5) / sqrt(0.25 / (N - 1)); // statistique normalisée (TCL)

    double p_value = erfc(fabs(z) / sqrt(2.0)); // p-value via loi normale

```

```

    int passed = (p_value >= 0.01);

    assert_test(passed, "Autocorrelation Test");
    ctr_drbg_uninstantiate(&ctx);
}
// Vérifie l'indépendance des bits voisins.
// Principe : on compare chaque bit avec le bit suivant (décalage = 1).
// On calcule le ratio de bits identiques à ce décalage.
// Si ce ratio est proche de 0.5, les bits ne dépendent pas fortement de leurs voisins.

// NIST SP 800-22 : "Frequency Test within a Block"
void test_block_frequency() {
    printf("\n--- Test Block Frequency ---\n");

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];
    memset(entropy, 0xAA, 48);

    ctr_drbg_init(&ctx, entropy, NULL);

    const size_t N = 200000; // nombre total de bits
    const size_t M = 128; // taille d'un bloc
    uint8_t out[N / 8];
    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    size_t num_blocks = N/M;

    double sum = 0.0;

    for (size_t b = 0; b < num_blocks; b++) {
        int ones = 0;
        // compter les 1 dans le bloc b
        for (size_t i = 0; i < M; i++) {
            int pos = b*M + i; // position globale du bit
            int byte = pos/8;
            int bit = pos%8;
            int value = (out[byte] >> bit) & 1;
            if (value == 1)
                ones++;
        }

        double pi_i = (double)ones/M; // proportion de 1 dans le bloc

        sum += (pi_i - 0.5)*(pi_i - 0.5); // NIST :  $X_i = p_i - 1/2$ 
    }

    double X = 4.0*M*sum; //  $X = \chi_{deux} = 4*M*somme((p_i - 0.5)^2)$ 

    double k = (double)num_blocks;
    double Z = (X - k)/sqrt(2.0 * k); // approximation du chi-deux par la loi normale
    double p_value = erfc(fabs(Z)/sqrt(2.0));

    int passed = (p_value >= 0.01); // seuil 1% (NIST)
    assert_test(passed, "Block Frequency Test ");

    ctr_drbg_uninstantiate(&ctx);
}
// Vérifie l'équilibre 0/1 sur de petits blocs de bits.
// Principe : on découpe la séquence en blocs (par ex. 128 bits) et on compte les 1.
// Si chaque bloc a un ratio de 1 proche de 0.5, la séquence est équilibrée même localement.

// NIST SP 800-22
void test_approximate_entropy() {
    printf("\n--- Approximate Entropy Test ---\n");

    const int n = 200000;
    const int m = 2; // on regarde les blocs de 2 bits
    /*m petit --> test rapide mais moins sensible
    m grand --> plus précis mais coûteux (exponentiel :  $2^m$ )
    On doit avoir  $m \leq \log_2(n)/5$ , ici on doit donc avoir  $m \leq 12$ 
    donc vaut mieux choisir  $m=2$  ou  $m=3$  (pas plus que  $m=4$  max) */

```

```

uint8_t out[n / 8];

CTR_DRBG_CTX ctx;
uint8_t entropy[48];

FILE *f = fopen("/dev/urandom", "rb");
fread(entropy, 1, 48, f);
fclose(f);

ctr_drbg_init(&ctx, entropy, NULL);
ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

int pow_m = 1 << m;
int pow_m1 = 1 << (m + 1);

double C_m[pow_m];
double C_m1[pow_m1];

memset(C_m, 0, sizeof(C_m));
memset(C_m1, 0, sizeof(C_m1));

for (int i = 0; i < n; i++) { // comptage des motifs (sequence circulaire)

    int v_m = 0;
    int v_m1 = 0;

    for (int j = 0; j < m; j++) { // motif taille m
        int bit = (out[(i + j) % n] / 8] >> ((i + j) % 8)) & 1;
        v_m = (v_m << 1) | bit;
    }

    for (int j = 0; j < m + 1; j++) { // motif taille m+1
        int bit = (out[(i + j) % n] / 8] >> ((i + j) % 8)) & 1;
        v_m1 = (v_m1 << 1) | bit;
    }

    C_m[v_m]++; // count m patterns
    C_m1[v_m1]++; // count m+1 patterns
}

double phi_m = 0.0; // phi(m)
double phi_m1 = 0.0; // phi(m+1)

for (int i = 0; i < pow_m; i++) {
    if (C_m[i] > 0) {
        double p = C_m[i] / n;
        phi_m += p * log(p); // contribution phi(m)
    }
}

for (int i = 0; i < pow_m1; i++) {
    if (C_m1[i] > 0) {
        double p = C_m1[i] / n;
        phi_m1 += p * log(p); // contribution phi(m+1)
    }
}

double ApEn = phi_m - phi_m1; // approximate entropy

double chi2 = 2.0 * n * (log(2.0) - ApEn);

double p_value = exp(-chi2 / 2.0); // approximation donc biais possible (pas comme dans NIST avec
↳ IGAMC)

int passed = (p_value >= 0.01);

assert_test(passed, "Approximate Entropy Test (NIST)");

ctr_drbg_uninstantiate(&ctx);
} // peut parfois échouer (normal)
// Vérifie la complexité de motifs courts (par ex. 2 bits).
// Principe : on compte combien de fois chaque motif de longueur m apparaît.
// On calcule ensuite l'entropie moyenne de ces motifs.
// Permet de détecter des répétitions locales dans la séquence.

```

```

//NIST SP 800-22
void test_cumulative_sums() {
    printf("\n--- Cumulative Sums Test) ---\n");

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];

    FILE *f = fopen("/dev/urandom", "rb");
    fread(entropy, 1, 48, f);
    fclose(f);

    ctr_drbg_init(&ctx, entropy, NULL);

    const size_t n = 200000;
    uint8_t out[n/8];
    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    int S = 0;
    int S_max_forward = 0; // on lit les bits de gauche à droite
    for (size_t i = 0; i < n; i++) { // i représente un indice dans un tableau (d'où le size_t)
        int bit = (out[i/8] >> (i%8)) & 1;
        int Xi = (bit == 1) ? 1 : -1; // notation NIST
        S += Xi;

        if (abs(S) > S_max_forward)
            S_max_forward = abs(S);
    }

    S = 0;
    int S_max_backward = 0; // on lit les bits de droite à gauche
    for (int i = n - 1; i >= 0; i--) { // int et pas size_t car besoin que i puisse être <0 (stop quand
        ↪ i=-1)

        int bit = (out[i / 8] >> (i % 8)) & 1;
        int Xi = (bit == 1) ? 1 : -1;
        S += Xi;

        if (abs(S) > S_max_backward)
            S_max_backward = abs(S);
    }

    int Z;
    if (S_max_forward > S_max_backward)
        Z = S_max_forward;
    else
        Z = S_max_backward;

    double Zn = (double)Z/sqrt(n);

    double p_value = erfc(Zn/sqrt(2.0));

    int passed = (p_value >= 0.01);
    assert_test(passed, "Cumulative Sums Test");
    ctr_drbg_uninstantiate(&ctx);
}

// Sert pour test linear complexity (voir lien source dans le mail)
int berlekamp_massey(int *s, int N) {

    int C[N], B[N], T[N]; // C = polynôme courant (modèle qui essaie de "prédire" la séquence)
    // B = ancien polynôme de référence, T = sauvegarde temporaire

    for (int i = 0; i < N; i++) { // initialisation
        C[i] = 0;
        B[i] = 0;
    }

    C[0] = 1; // au départ : C(x)=1 et B(x)=1
    B[0] = 1;

    int L = 0; // longueur du modèle (complexité linéaire)
    int m = -1; // position du dernier changement important

    for (int n = 0; n < N; n++) {

```

```

// calcul de d = erreur de prédiction (discrepancy)
int d = s[n];

for (int i = 1; i <= L; i++) { // on teste si le modèle actuel prédit correctement s[n]
    d ^= (C[i] & s[n - i]);
}

if (d == 1) { // si d = 0 : le modèle explique bien le bit, si d=1 : il se trompe

    // sauvegarde C
    for (int i = 0; i < N; i++) // on sauvegarde l'ancien modèle
        T[i] = C[i];

    // C(x) = C(x) XOR x^(n-m) * B(x)
    for (int i = 0; i < N - (n - m); i++) { // on corrige le modèle en ajoutant une nouvelle
        ↪ dépendance
        C[i + (n - m)] ^= B[i];
    }

    if (2 * L <= n) { // si le modèle devient insuffisant
        L = n + 1 - L; // on augmente la complexité
        m = n; // on mémorise le point de correction

        for (int i = 0; i < N; i++) B[i] = T[i]; // on met à jour le modèle de référence
    }
}

return L; // complexité linéaire finale
}

// NIST SP 800-22
/* mesure la complexité linéaire d'une séquence binaire, c-à-d
la taille du plus petit registre à décalage linéaire qui peut générer la séquence */
void test_linear_complexity() {

    printf("\n--- Linear Complexity Test ---\n");

    const int n = 200000; // nb bits
    const int M = 500; // taille des blocs
    /*Le NIST choisit typiquement : M=500 car :
assez grand pour que Berlekamp-Massey soit significatif
assez petit pour avoir assez de blocs
règle : N = n/M >= 200 donc on veut au moins 200 blocs*/

    uint8_t out[n/8];

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];

    FILE *f = fopen("/dev/urandom", "rb");
    fread(entropy, 1, 48, f);
    fclose(f);

    ctr_drbg_init(&ctx, entropy, NULL);
    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    int N = n/M; // nb blocs

    double chi2 = 0.0;

    for (int b = 0; b < N; b++) {

        int s[M];

        for (int i = 0; i < M; i++) { // conversion bloc en tableau de bits
            int pos = b*M + i;
            int bit = (out[pos/8] >> (pos%8)) & 1;
            s[i] = bit;
        }

        int L_i = berlekamp_massey(s, M); // L_i = complexité linéaire réelle (Berlekamp-Massey)
    }
}

```

```

    double mu = M/2.0; // moyenne théorique NIST (simplifiée) ici pas correction périodique exacte des
    ↪ bins NIST (car trop lourd)

    double T = L_i - mu; // écart à la moyenne

    chi2 += T*T; // accumulation pour le chi-deux
}

double X = (4.0/M)*chi2; // statistique NIST simplifiée

double p_value = exp(-X / 2.0); // approximation simple chi-deux (pas IGAMMC du NIST)

int passed = (p_value >= 0.01); // seuil NIST 1%

assert_test(passed, "Linear Complexity Test");

ctr_drbg_uninstantiate(&ctx);
}
// Vérifie la complexité linéaire de la séquence.
// Principe : on applique l'algorithme de Berlekamp-Massey pour calculer le plus petit registre à décalage
// capable de générer la séquence.
// Si la complexité est proche de N/2, la séquence est considérée complexe et aléatoire.

// NIST SP 800-22 : "Discrete Fourier Transform (Spectral) Test"
// détecte des motifs périodiques dans la séquence en utilisant la transformée de Fourier discrète :
void test_spectral() {

    printf("\n--- Spectral Test ---\n");

    const int n = 20000;

    uint8_t out[n/8];

    CTR_DRBG_CTX ctx;
    uint8_t entropy[48];

    FILE *f = fopen("/dev/urandom", "rb");
    fread(entropy, 1, 48, f);
    fclose(f);

    ctr_drbg_init(&ctx, entropy, NULL);
    ctr_drbg_generate(&ctx, out, sizeof(out), NULL);

    double complex X[n];

    for (int i = 0; i < n; i++) {
        int bit = (out[i / 8] >> (i % 8)) & 1;
        X[i] = (bit ? 1 : -1) + 0.0 * I; // X_i = +ou-1
    }

    // DFT naïve ~ O(n^2) (plus lent que FFT ~ O(n*log(n)))
    int half = n/2;
    double M[half];
    for (int k = 0; k < half; k++) {
        double complex S = 0;
        for (int j = 0; j < n; j++) {
            S += X[j]*cexp(-2.0*M_PI*I*k*j/n); // cexp = exponentielle complexe
        }
        M[k] = cabs(S); //cabs = valeur absolue complexe
    }

    double T = sqrt(log(1/0.05)*n); // seuil NIST

    int N1 = 0;
    for (int k = 0; k < half; k++) {
        if (M[k] < T)
            N1++;
    }

    double N0 = 0.95 * half; // valeur attendue

    double d =(N1 - N0)/sqrt(n*0.95*0.05/4.0);

    double p_value = erfc(fabs(d)/sqrt(2.0));

```

```

    int passed = (p_value >= 0.01);

    assert_test(passed, "Spectral Test");

    ctr_drbg_uninstantiate(&ctx);
}
// Vérifie l'absence de motifs périodiques.
// Principe : on transforme la séquence binaire en +1/-1 et on applique la DFT.
// On regarde les amplitudes des fréquences, si certaines dépassent un seuil trop souvent,
// cela indique des motifs répétitifs.

// test des timing entre notre ctr_drbg et le générateur de linux (/dev/urandom) :

static double diff_time(struct timespec a, struct timespec b) {
    return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / 1e9; // conversion en secondes
}

// calcule un chi-square simple sur les bytes
double chi_square(uint8_t *data, size_t n) {

    int freq[256] = {0}; // tableau des fréquences pour chaque valeur de byte

    for (size_t i = 0; i < n; i++)
        freq[data[i]]++; // compte combien de fois chaque byte apparaît

    double expected = n / 256.0; // valeur attendue si distribution uniforme
    double chi = 0;

    for (int i = 0; i < 256; i++) {
        double d = freq[i] - expected; // écart à la moyenne
        chi += (d * d) / expected; // contribution chi-square
    }

    return chi; // plus petit = plus uniforme
}

// uniformité simple : pire écart à 1/256
double max_deviation(uint8_t *data, size_t n) {

    int freq[256] = {0};

    for (size_t i = 0; i < n; i++)
        freq[data[i]]++;

    double max_dev = 0;

    for (int i = 0; i < 256; i++) {
        double p = (double)freq[i] / n;
        double d = fabs(p - 1.0 / 256.0);

        if (d > max_dev)
            max_dev = d;
    }

    return max_dev;
}

void test_comparaison() {

    printf("\n--- Comparison CTR_DRBG vs Linux RNG ---\n");

    const size_t N = 1000000; // taille des données générées (1 MB)

    uint8_t *ctr = malloc(N); // buffer
    uint8_t *gr = malloc(N);
    uint8_t *urandom = malloc(N);

    if (!ctr || !gr || !urandom) {
        printf("memory error\n");
        return;
    }

    // CTR_DRBG :

```

```

CTR_DRBG_CTX ctx;
uint8_t entropy[48]; // seed initial (entropie)

FILE *f = fopen("/dev/urandom", "rb");
if (!f) {
    perror("fopen");
    exit(1);
}

size_t r = fread(entropy, 1, sizeof(entropy), f);
if (r != sizeof(entropy)) {
    fprintf(stderr, "entropy read failed\n");
    fclose(f);
    exit(1);
}

fclose(f);

ctr_drbg_init(&ctx, entropy, NULL);

struct timespec t1, t2;

clock_gettime(CLOCK_MONOTONIC, &t1); // début du chronomètre
ctr_drbg_generate(&ctx, ctr, N, NULL);
clock_gettime(CLOCK_MONOTONIC, &t2); // fin du chronomètre

ctr_drbg_uninstantiate(&ctx); // nettoyage de l'état interne

double time_ctr = diff_time(t1, t2); // temps de génération CTR_DRBG

// getrandom :

clock_gettime(CLOCK_MONOTONIC, &t1);
if (getrandom(gr, N, 0) != (ssize_t)N) { // génération via kernel Linux
    perror("getrandom");
    exit(1);
}
clock_gettime(CLOCK_MONOTONIC, &t2);

double time_gr = diff_time(t1, t2);

// /dev/urandom (ancien mais encore utilisé) :

FILE *fu = fopen("/dev/urandom", "rb");
if (!fu) {
    perror("fopen /dev/urandom");
    exit(1);
}

clock_gettime(CLOCK_MONOTONIC, &t1);
size_t r2 = fread(urandom, 1, N, fu);
clock_gettime(CLOCK_MONOTONIC, &t2);

if (r2 != N) {
    fprintf(stderr, "urandom read failed\n");
    fclose(fu);
    exit(1);
}

fclose(fu);

double time_ur = diff_time(t1, t2);

// QUALITY :

double chi_ctr = chi_square(ctr, N); // uniformité globale CTR_DRBG
double chi_gr = chi_square(gr, N); // uniformité de getrandom
double chi_ur = chi_square(urandom, N); // uniformité de /dev/urandom

double dev_ctr = max_deviation(ctr, N); // biais maximal CTR_DRBG
double dev_gr = max_deviation(gr, N); // biais maximal getrandom
double dev_ur = max_deviation(urandom, N); // biais maximal /dev/urandom

```



```

// RESULTS :

printf("\n--- SPEED ---\n"); // performance (temps)

printf("CTR_DRBG      : %.4f s\n", time_ctr);
printf("getrandom()    : %.4f s\n", time_gr);
printf("/dev/urandom    : %.4f s\n", time_ur);

printf("\n--- CHI-SQUARE ---\n"); // qualité globale

printf("CTR_DRBG      : %.2f\n", chi_ctr);
printf("getrandom()    : %.2f\n", chi_gr);
printf("/dev/urandom    : %.2f\n", chi_ur);

printf("\n--- UNIFORMITY (max deviation) ---\n"); // biais local

printf("CTR_DRBG      : %.6f\n", dev_ctr); // pire écart CTR_DRBG
printf("getrandom()    : %.6f\n", dev_gr); // pire écart Linux moderne
printf("/dev/urandom    : %.6f\n", dev_ur); // pire écart Linux ancien

free(ctr);
free(gr);
free(urandom);
}

// Programme Principal : Rapport de test

int main() {
    printf("    STARTING CTR_DRBG TEST SUITE    \n");

    // Tests fonctionnels / conformité
    test_uninstantiate();
    test_constraints();
    test_known_answers();

    // Tests statistiques
    test_frequency();
    test_runs();
    test_autocorrelation(); // détecte la dépendance entre bits voisins (complète test_runs)
    test_block_frequency();
    test_approximate_entropy();
    test_cumulative_sums(); // similaire au Random Walk test du NIST, vérifie les déséquilibres cumulatifs
    test_linear_complexity();
    test_spectral();

    // Test de non-régression
    // Répéter le KAT pour s'assurer du non-changement
    test_known_answers();

    printf("\n");
    printf("                FINAL REPORT                \n");
    printf("Tests executed   : %d\n", tests_run);

    if (tests_failed == 0) {
        printf("Tests passed     : %d (100%)\n", tests_passed);
        printf(" \n>>> ALL TESTS PASSED SUCCESSFULLY! <<< \n");
    } else {
        printf("Tests passed     : %d\n", tests_passed);
        printf("Tests failed     : %d\n", tests_failed);
        printf(" \n>>> WARNING: SOME TESTS FAILED <<< \n");
    }
    printf("\n");
    // test de comparaison
    test_comparaison();
    return (tests_failed == 0) ? EXIT_SUCCESS : EXIT_FAILURE;
}

```